

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Multicores and Multiprogramming (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Dennard scaling and Moore's law
- Types of parallelism
- Parallelism and concurrency

This Lecture

- Parallel computation complexity
- Flynn's Taxonomy
- Multithreading
- Parallel programming

Computation Complexity

1. Model an algorithm as a set of steps/components, which represent units of some evaluation metric (e.g., time, storage space).
2. Determine the number of steps/components required by the algorithm as a function of input size.

Example:

Given an input array of size N ,

- Merge sort takes $O(N \log N)$ time
- Bubble sort takes $O(N^2)$ time

Parallel Computation Complexity

Extend to parallel algorithms:

- T_1 : Time on single processor core
- T_p : Time on P processor cores
- T_∞ : Time on infinitely many processor cores (ideal)

...and $T_p \geq T_\infty$

Parallel Computation Complexity

Extend to parallel algorithms:

- T_1 : Time on single processor core
- T_p : Time on P processor cores
- T_∞ : Time on infinitely many processor cores (ideal)

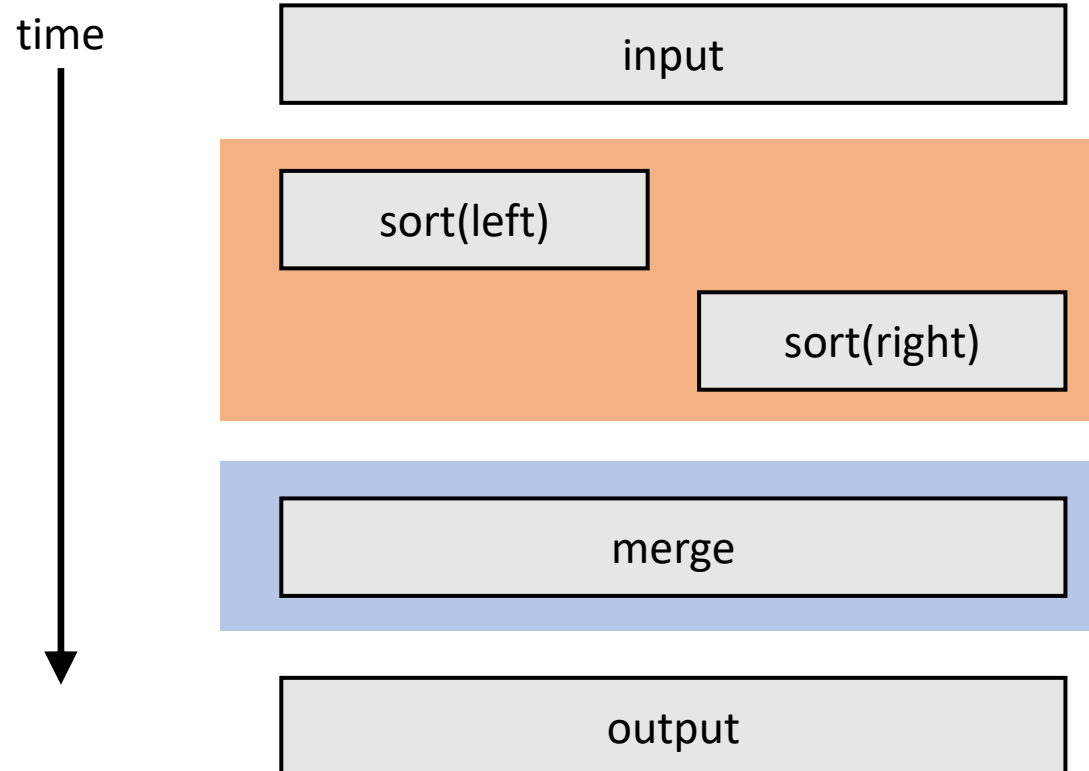
...and $T_p \geq T_\infty$

Work: Total number of steps in algorithm (T_1)

Depth: Longest chain of dependent steps (T_p or T_∞)

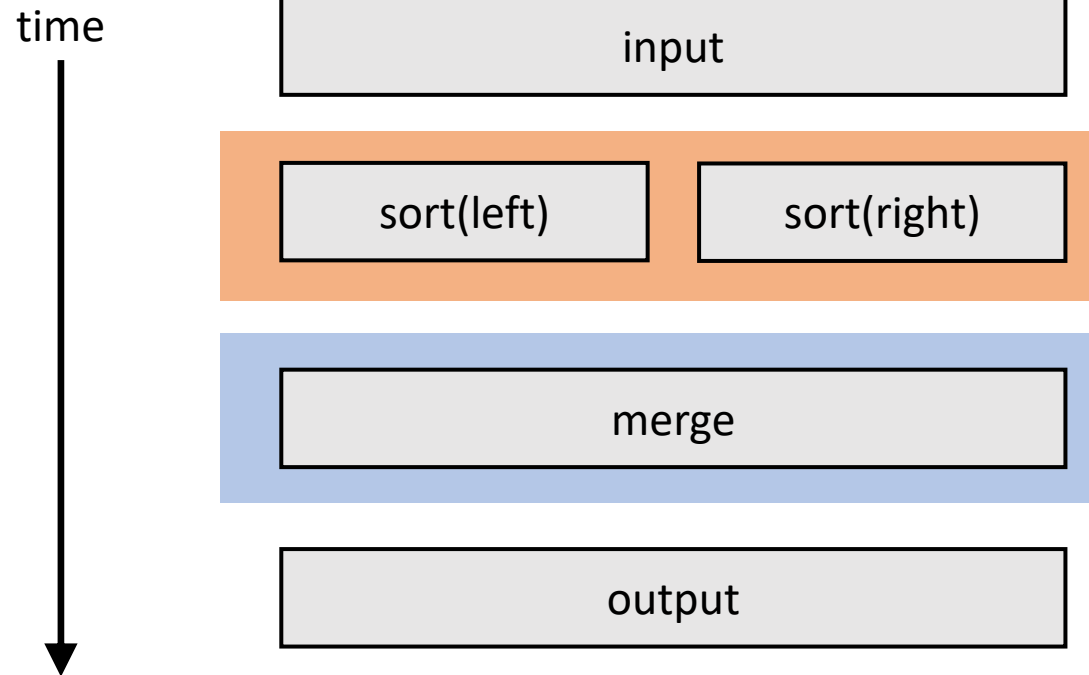
Parallel Computation Complexity

Merge Sort Example: Sequential execution



Parallel Computation Complexity

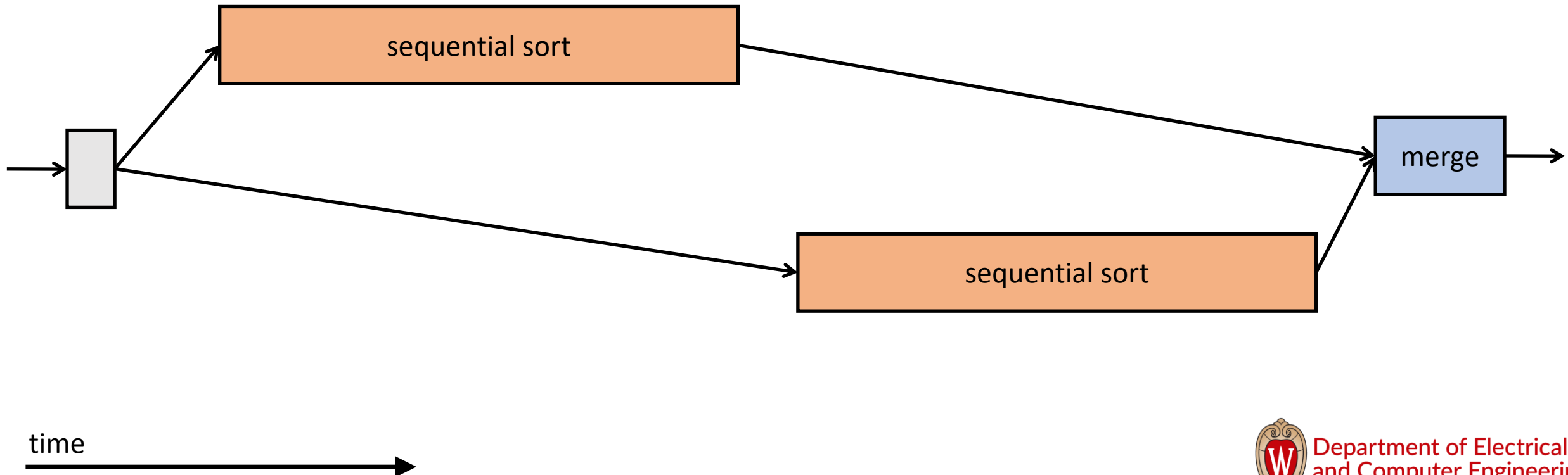
Merge Sort Example: Parallel execution



Parallel Computation Complexity

Merge Sort Example:

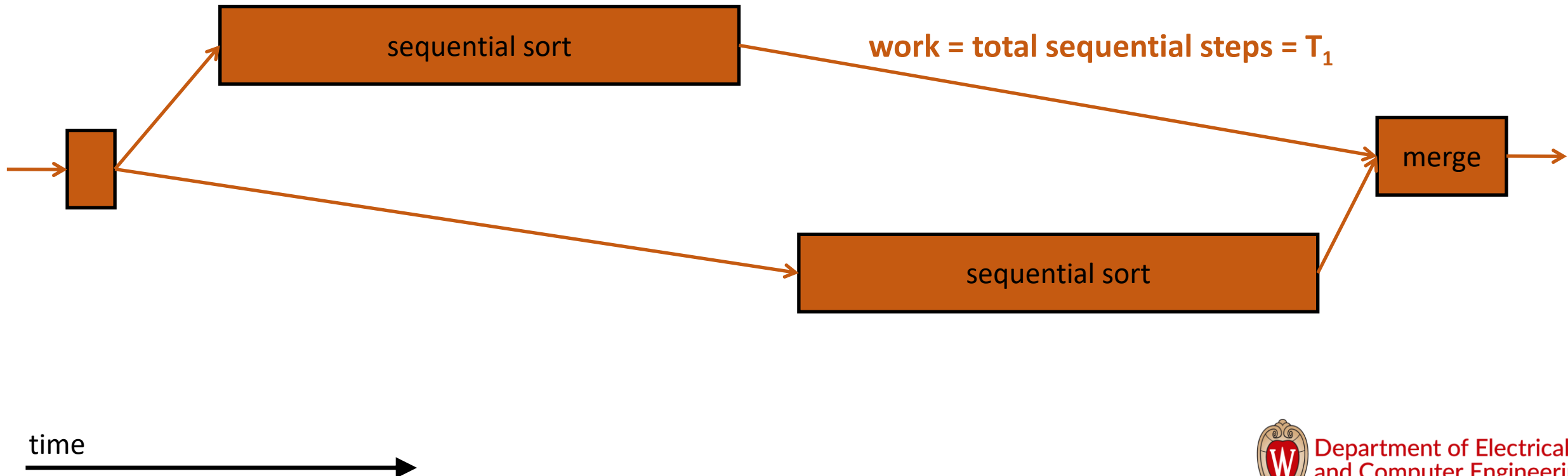
Sequential execution (1 core)



Parallel Computation Complexity – Work

Merge Sort Example:

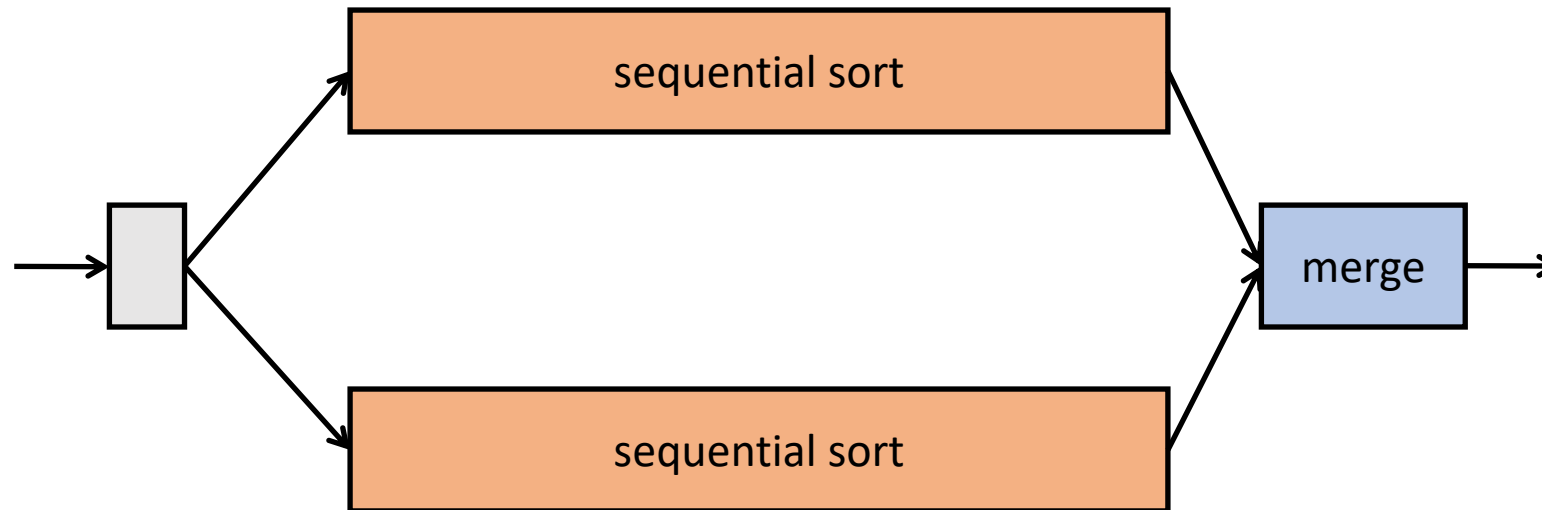
Sequential execution (1 core)



Parallel Computation Complexity

Merge Sort Example:

Parallel execution (2 cores)

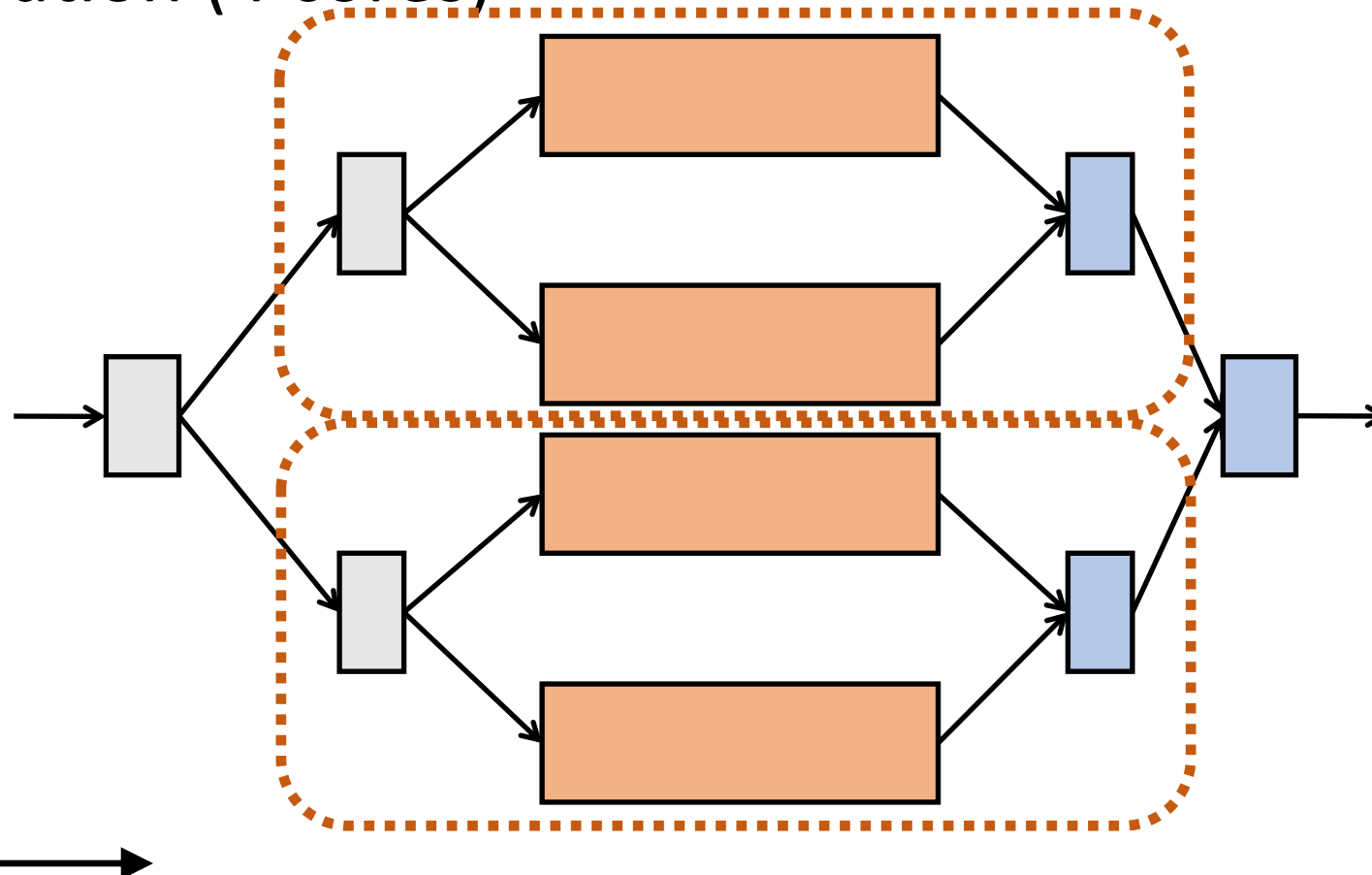


time →

Parallel Computation Complexity

Merge Sort Example:

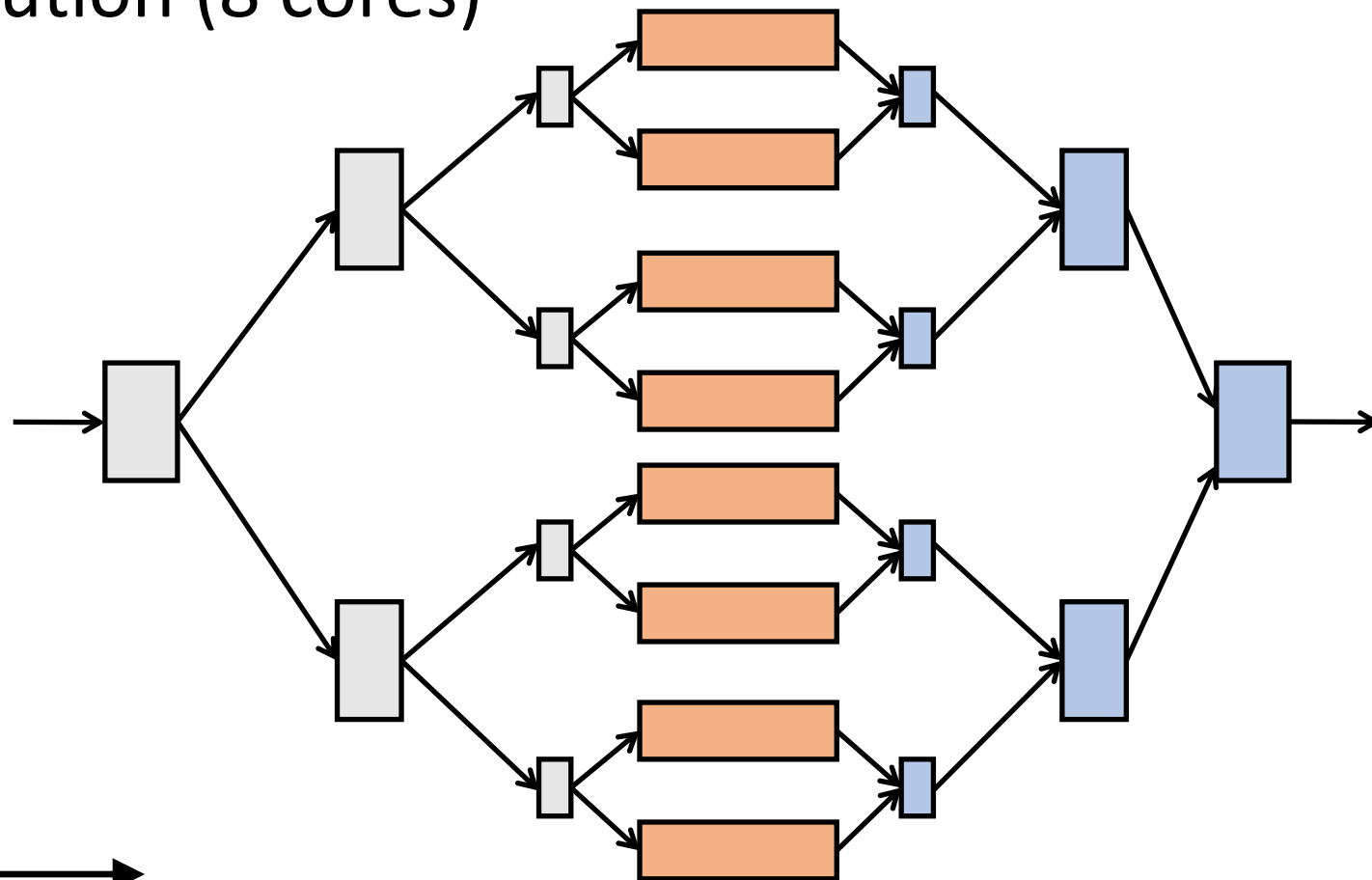
Parallel execution (4 cores)



Parallel Computation Complexity

Merge Sort Example:

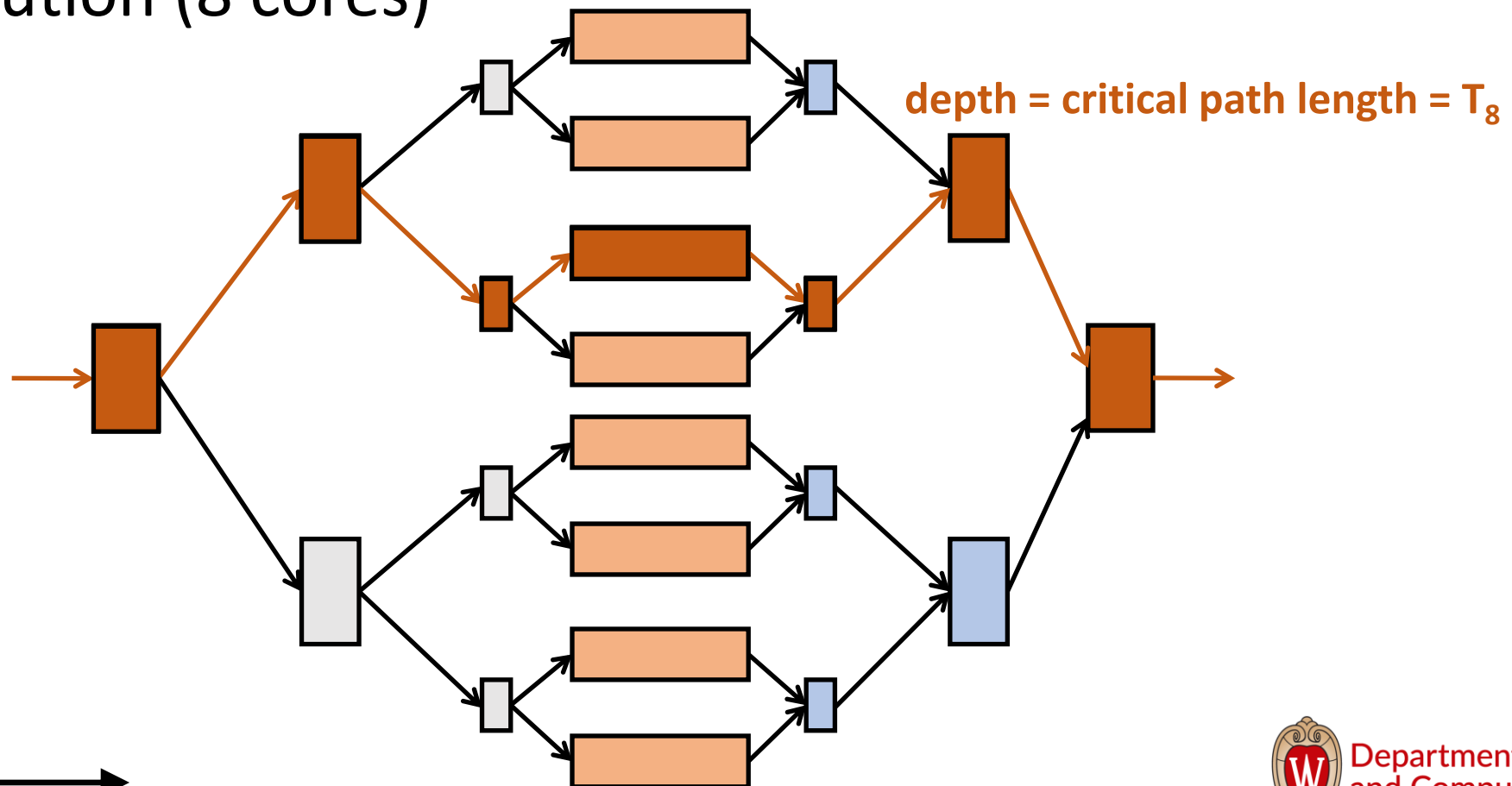
Parallel execution (8 cores)



Parallel Computation Complexity – Depth

Merge Sort Example:

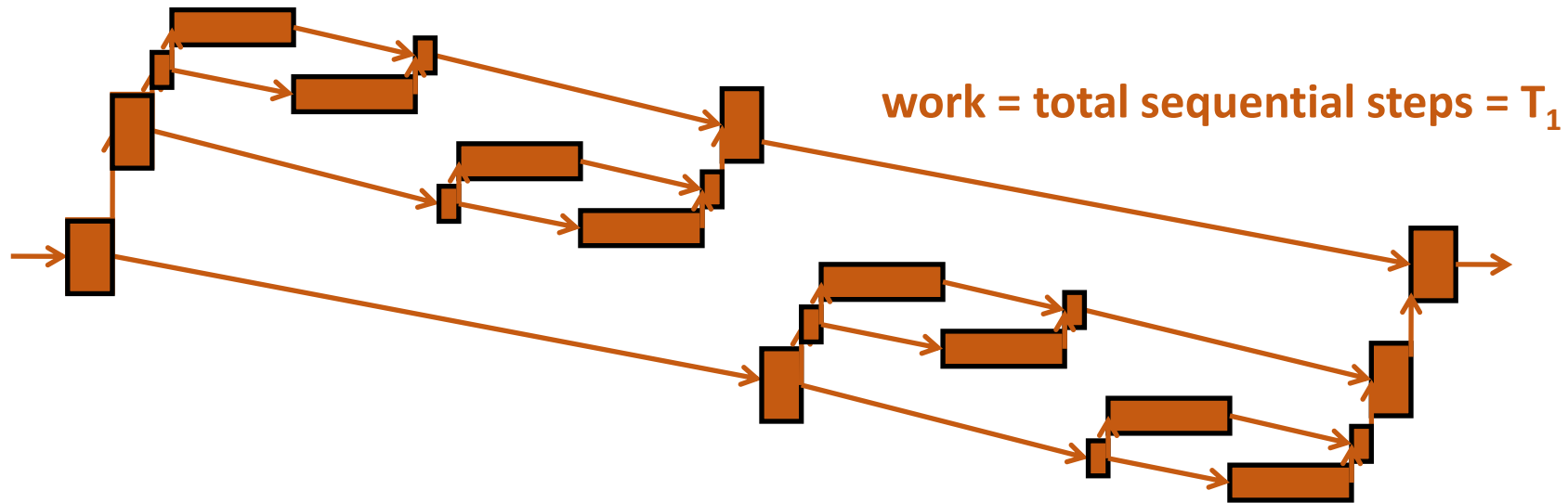
Parallel execution (8 cores)



Parallel Computation Complexity – Work

Merge Sort Example:

Parallel execution (8 cores)



Parallel Computation Complexity

P-core parallel speedup = work / depth:

$$S_p = T_1 / T_p$$

Ideal parallel speedup = work / depth:

$$S_\infty = T_1 / T_\infty$$

...and $S_p \leq S_\infty$

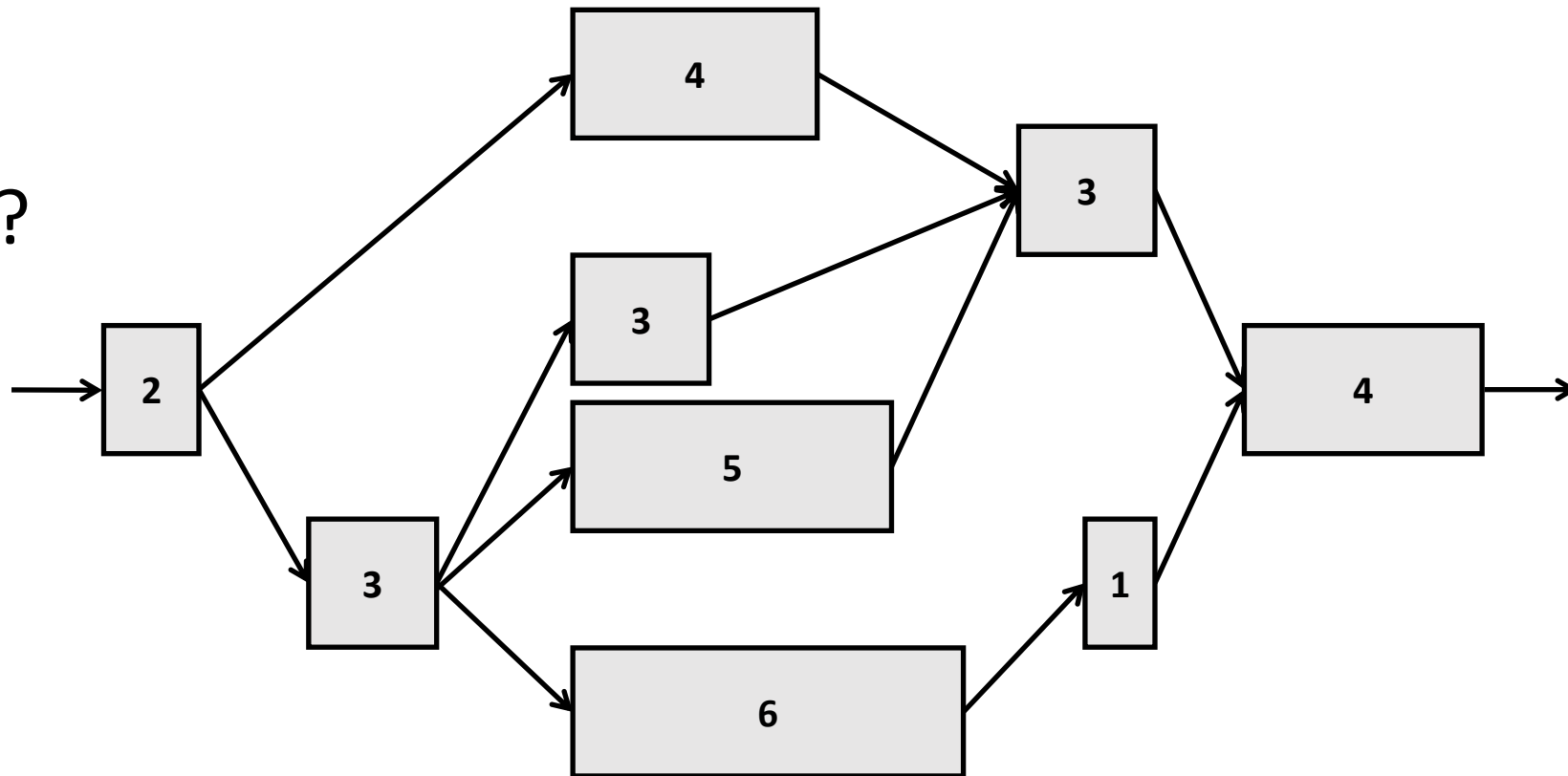
Parallel Computation Complexity

Example:

Work = ?

Depth = ?

Speedup = ?



time →

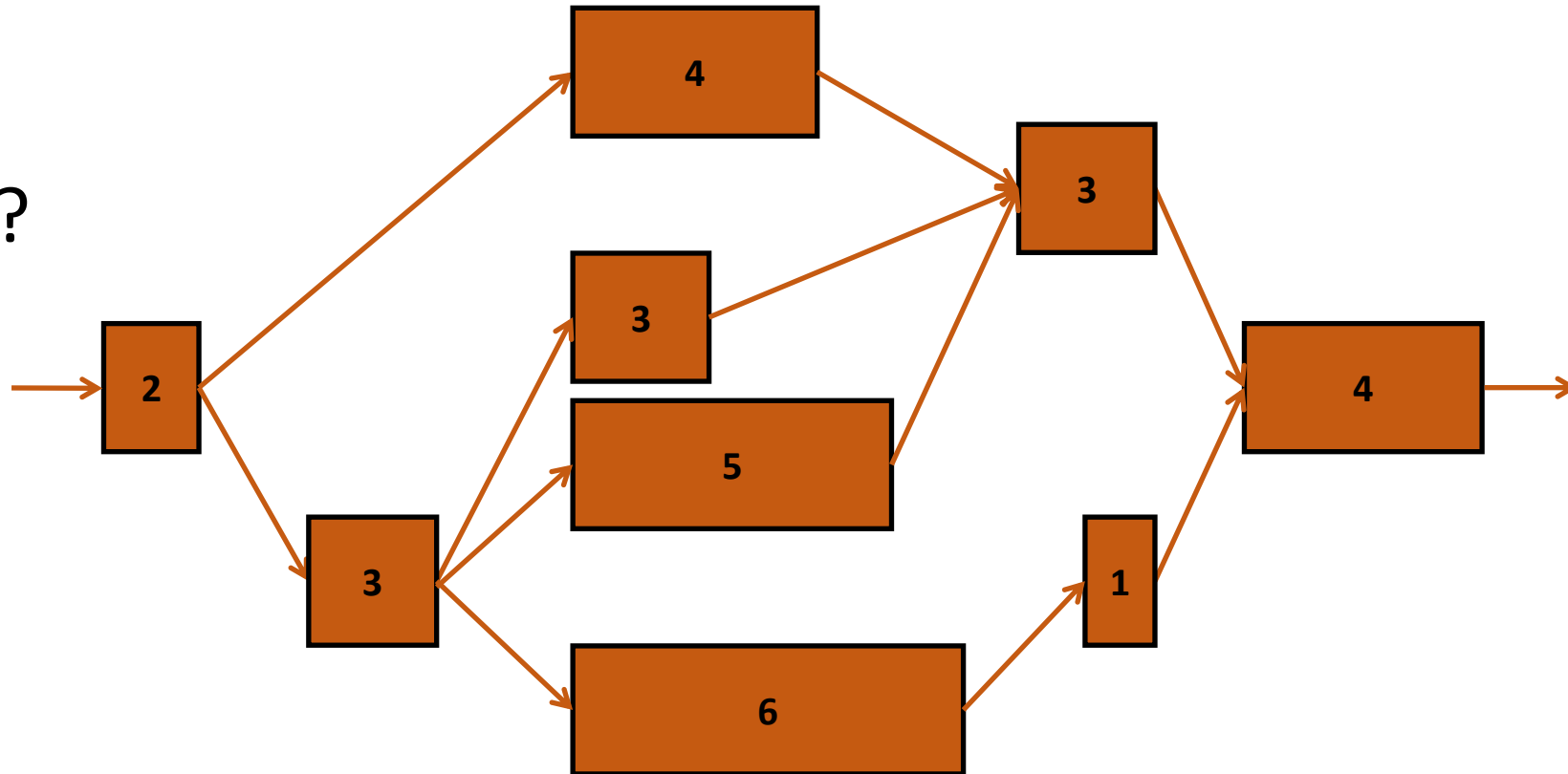
Parallel Computation Complexity

Example:

Work = 31

Depth = ?

Speedup = ?



time →

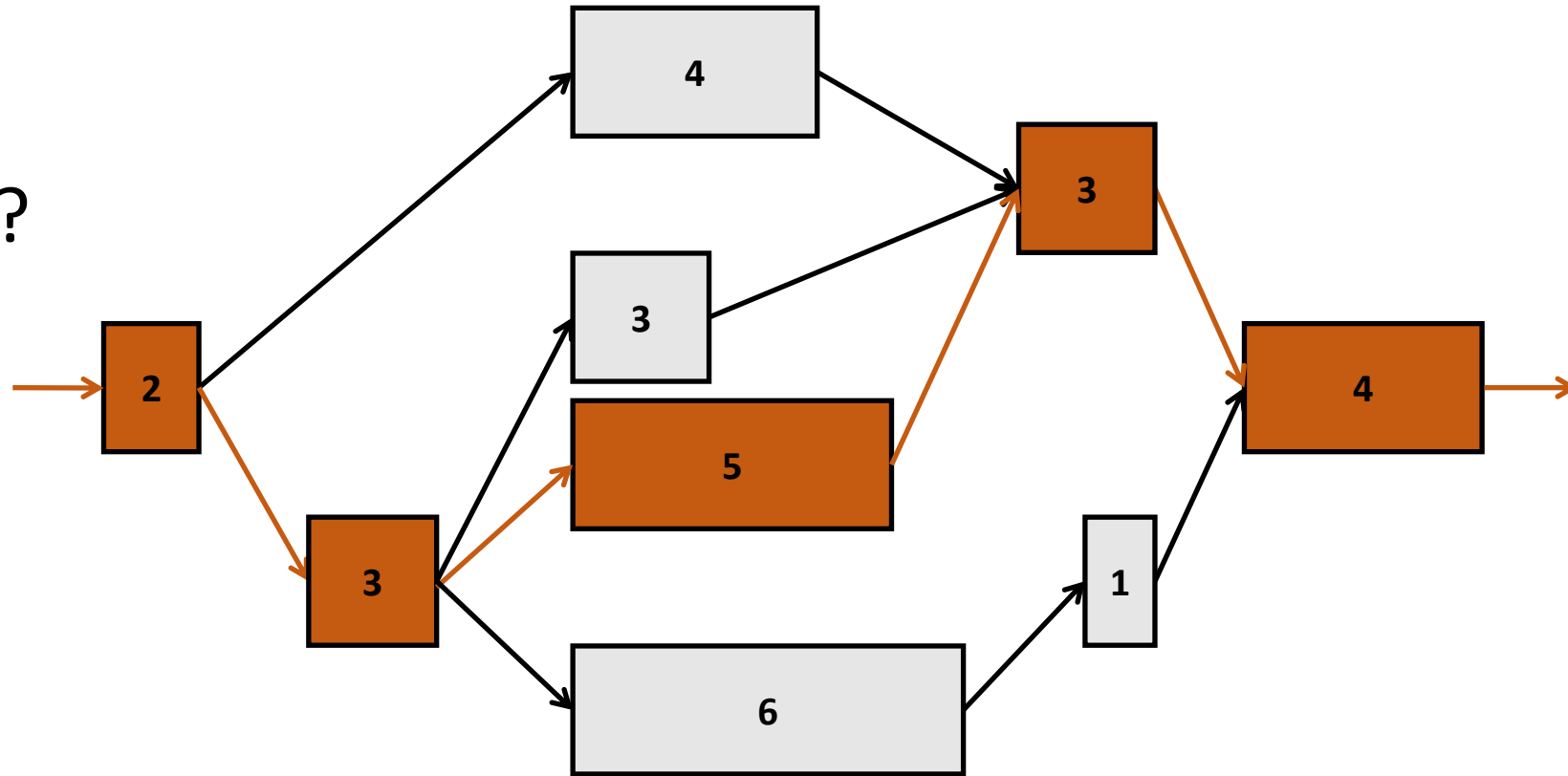
Parallel Computation Complexity

Example:

Work = 31

Depth = 17

Speedup = ?



time →

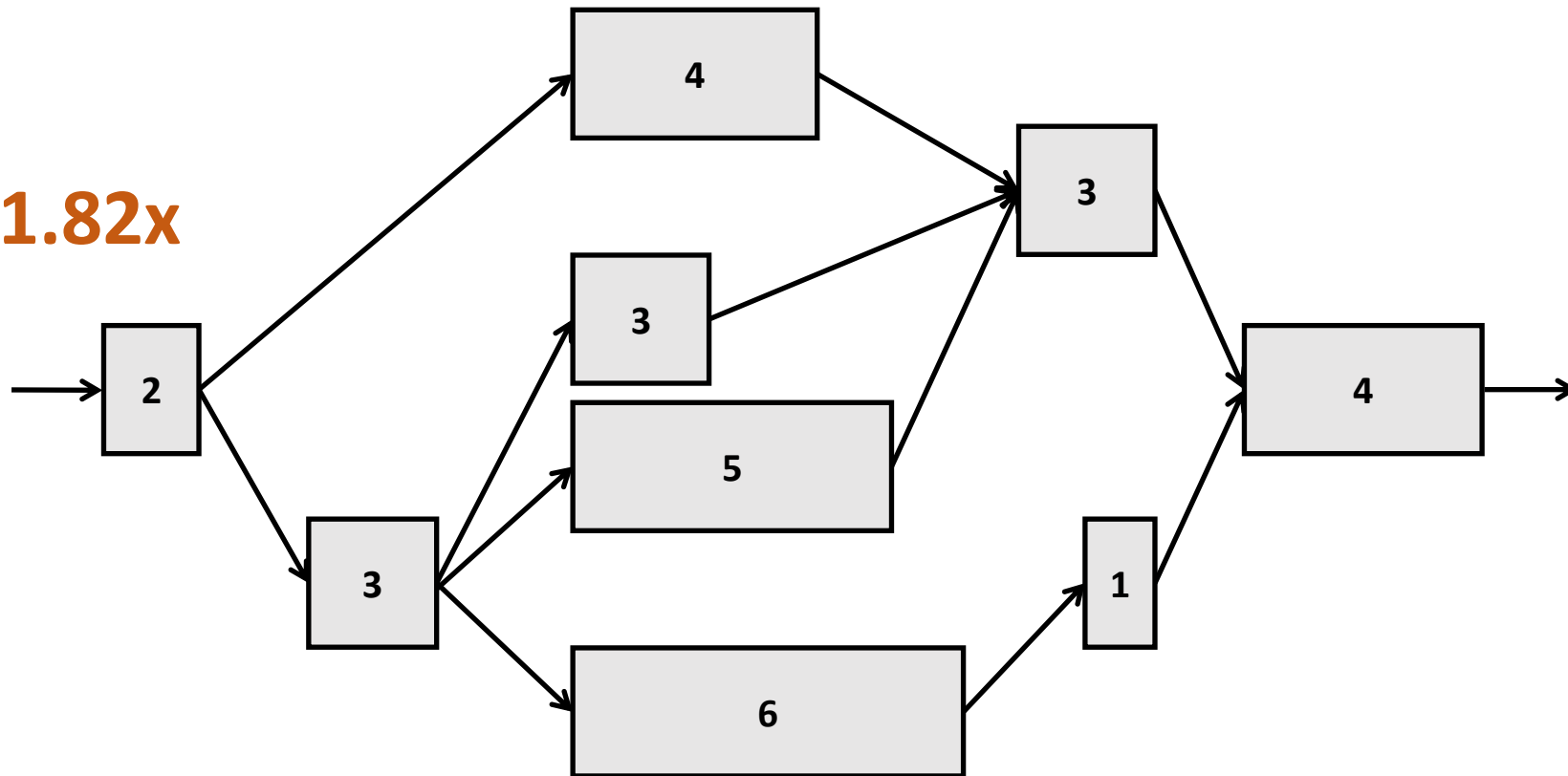
Parallel Computation Complexity

Example:

Work = 31

Depth = 17

Speedup = 1.82x



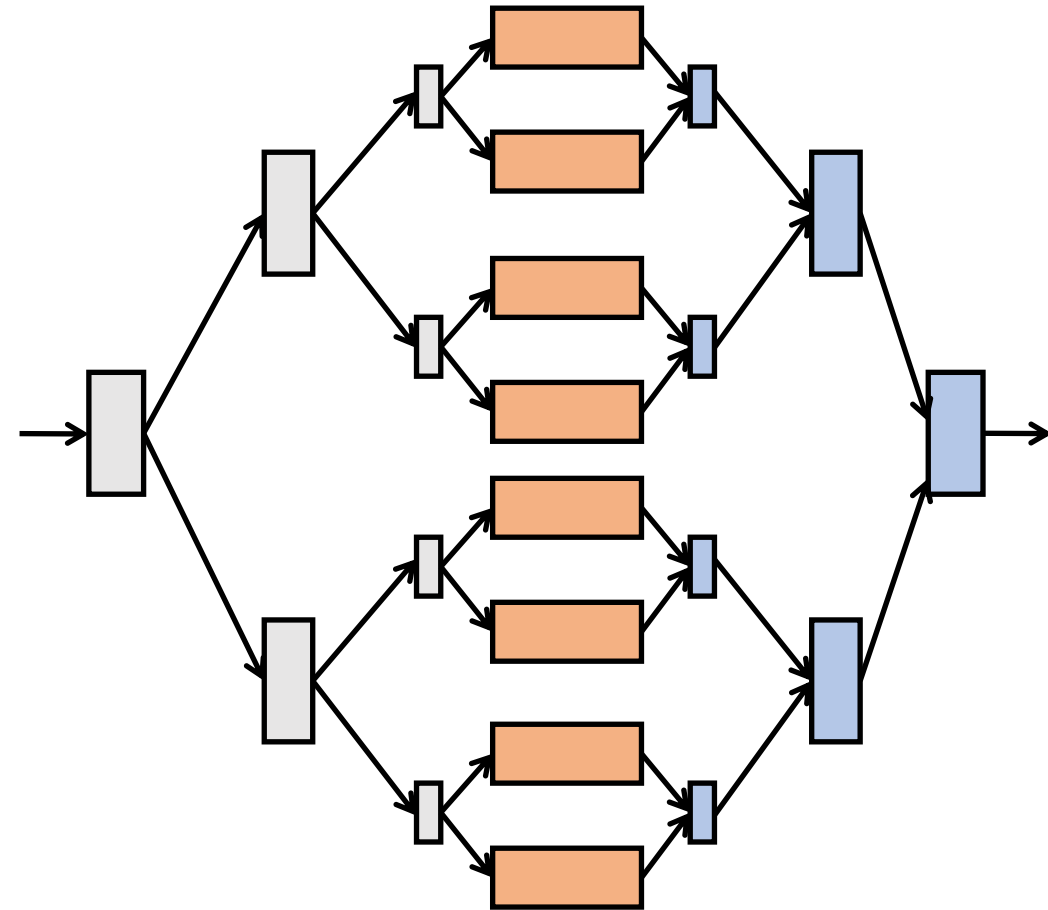
time
→

Parallel Computation Complexity

Merge Sort Example:

Given input size N ,

- $\text{Work} = T_1 = O(N \cdot \log N)$
- $\text{Depth} = T_\infty = O(N)$
- It takes $O(N)$ time to merge N elements



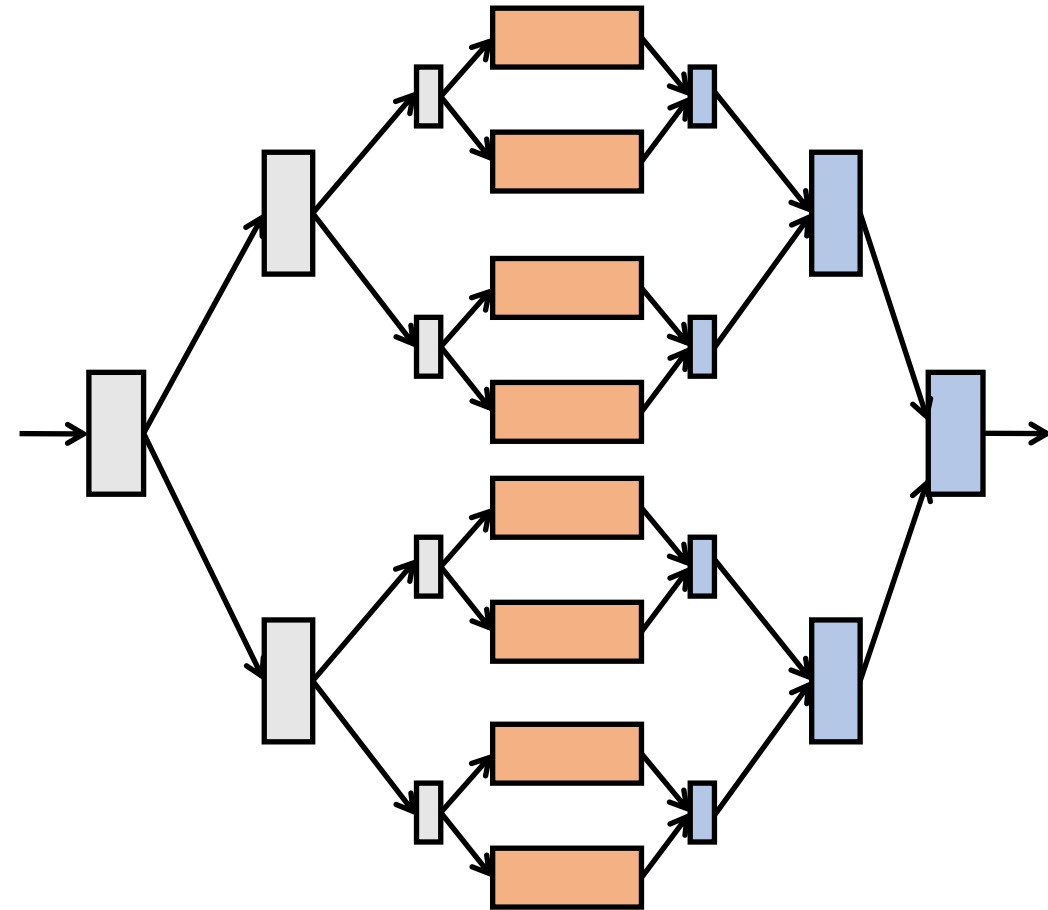
Parallel Computation Complexity

Merge Sort Example:

Given input size N ,

- Work = $T_1 = O(N \cdot \log N)$
- Depth = $T_\infty = O(N)$
- It takes $O(N)$ time to merge N elements

Parallel Speedup: $T_1 / T_\infty \sim \log N$



Parallel Computation Complexity

For sequential performance:

Any reduction in computation is beneficial

- Minimize work

For parallel performance:

Target the critical path (sequential bottleneck)

- Minimize depth

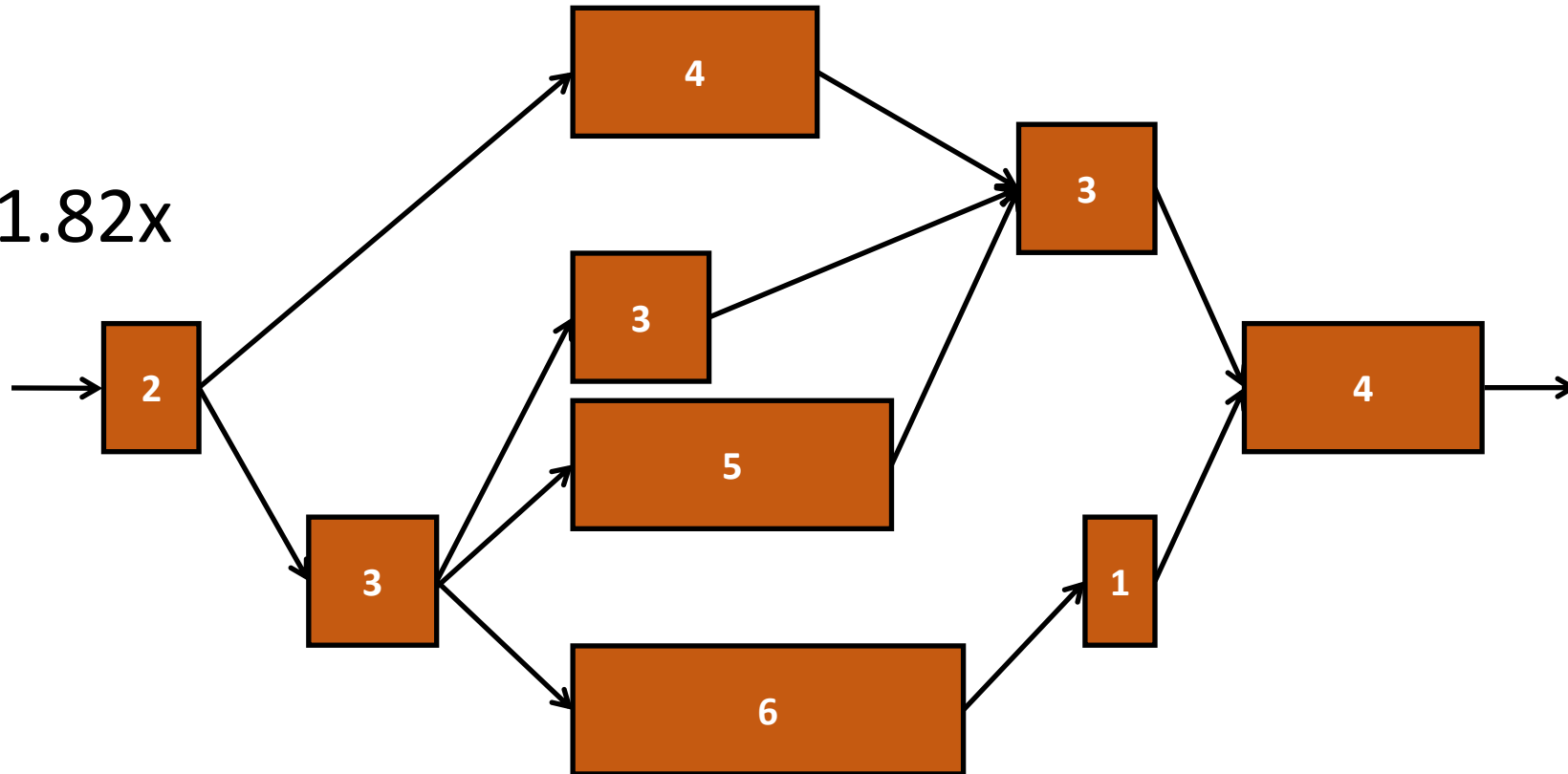
Parallel Speedup

Example:

Work = 31

Depth = 17

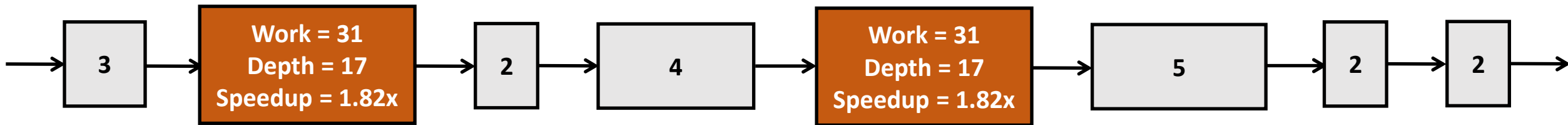
Speedup = 1.82x



time →

Parallel Speedup

Example:

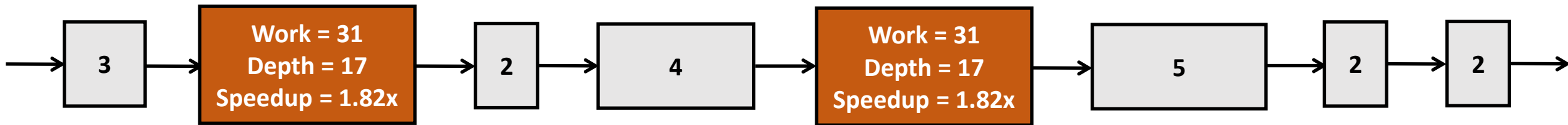


time
→

Parallel Speedup

Example:

Application Speedup = AS = ?

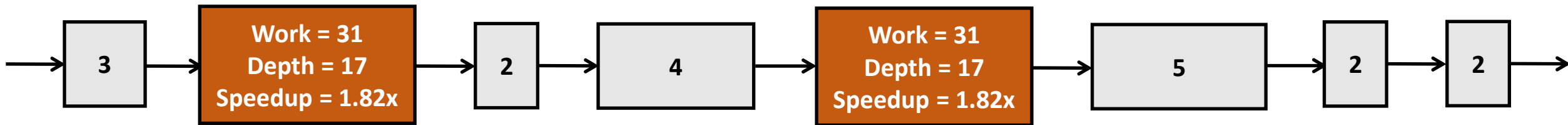


time
→

Parallel Speedup

Example:

Application Speedup = AS = ?



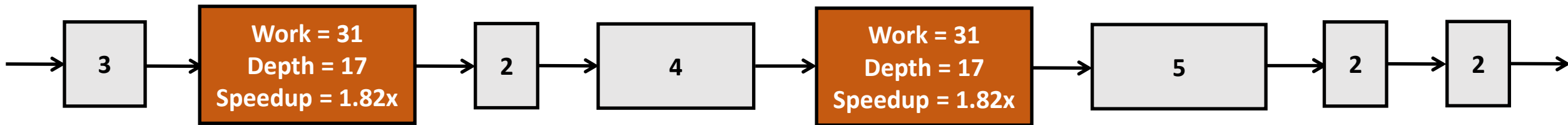
$$AS = \frac{\text{Application Work}}{\text{Application Depth}} = \frac{3 + 31 + 2 + 4 + 31 + 5 + 2 + 2}{3 + 17 + 2 + 4 + 17 + 5 + 2 + 2} = 1.54x$$

time
→

Parallel Speedup

Example:

Application Speedup = AS = ?



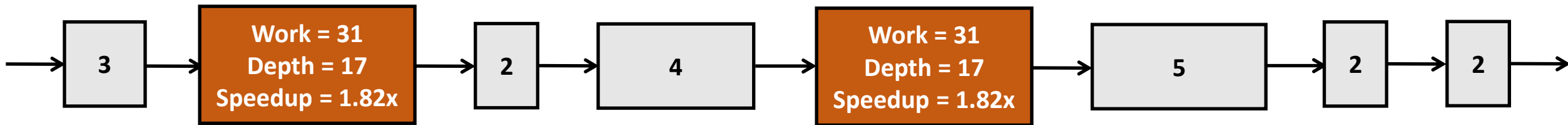
$$AS = \frac{\text{Application Work}}{\text{Application Depth}} = \frac{\overset{\text{sequential work}}{(3 + 2 + 4 + 5 + 2 + 2)} + \overset{\text{parallel work}}{(31 + 31)}}{\underset{\text{sequential depth}}{(3 + 2 + 4 + 5 + 2 + 2)} + \underset{\text{parallel depth}}{(17 + 17)}} = 1.54x$$

time
→

Parallel Speedup

Example:

Application Speedup = AS = ?



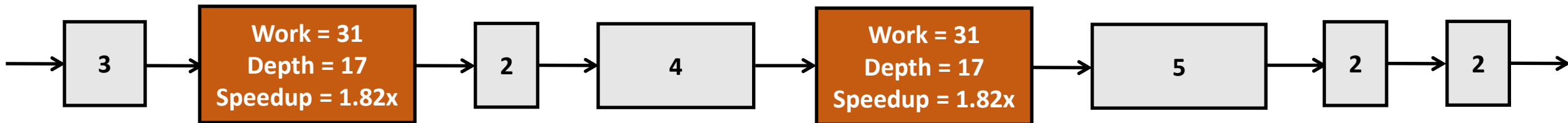
$$AS = \frac{\text{Application Work}}{\text{Application Depth}} = \frac{(3 + 2 + 4 + 5 + 2 + 2) + (31 + 31)}{(3 + 2 + 4 + 5 + 2 + 2) + \frac{(31 + 31)}{1.82}} = 1.54x$$

time
→

Parallel Speedup

Example:

Application Speedup = AS = ?



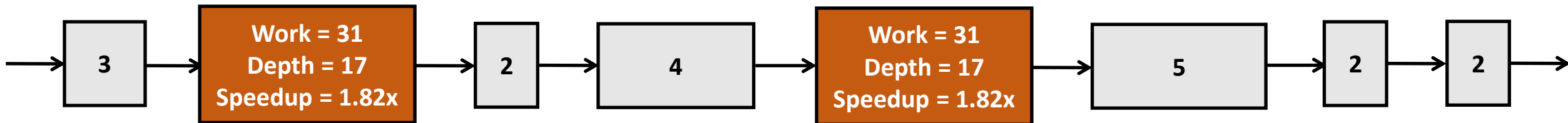
$$AS = \frac{\frac{\text{Application Work}}{\text{Application Depth}}}{\frac{\text{Application Work}}{\text{Application Speedup}}} = \frac{(3 + 2 + 4 + 5 + 2 + 2) + (31 + 31)}{(3 + 2 + 4 + 5 + 2 + 2) + \frac{(31 + 31)}{1.82}} = 1.54x$$

time
→

Parallel Speedup

Example:

Application Speedup = AS = ?



$$AS = \frac{\text{Application Work}}{\text{Application Depth}} = \frac{1}{\%_{\text{sequential}} + \frac{\%_{\text{parallel}}}{1.82}} = 1.54x$$

time
→

Amdahl's Law

Given sequential and parallel fractions of application work:

$$\text{Application speedup, } P \text{ cores} = AS_P = \frac{1}{\%_{\text{sequential}} + \frac{\%_{\text{parallel}}}{S_P}}$$

$$\text{Application speedup, ideal} = AS_{\infty} = \frac{1}{\%_{\text{sequential}} + \frac{\%_{\text{parallel}}}{S_{\infty}}}$$

Amdahl's Law – Upper Bound Speedup

Given sequential and parallel fractions of application work:

$$\textit{Upper bound speedup, } P \textit{ cores} = AS_P = \frac{1}{\%_{\textit{sequential}} + \frac{\%_{\textit{parallel}}}{P}}$$

$$\textit{Upper bound speedup, ideal} = AS_{\infty} = \frac{1}{\%_{\textit{sequential}} + \frac{\%_{\textit{parallel}}}{\infty}}$$

Amdahl's Law – Upper Bound Speedup

Given sequential and parallel fractions of application work:

$$\textit{Upper bound speedup, } P \textit{ cores} = AS_P = \frac{1}{\%_{\textit{sequential}} + \frac{\%_{\textit{parallel}}}{P}}$$

Can S_P ever exceed P in practice?

$$\textit{Upper bound speedup, ideal} = AS_{\infty} = \frac{1}{\%_{\textit{sequential}} + \frac{\%_{\textit{parallel}}}{\infty}}$$



Amdahl's Law – General

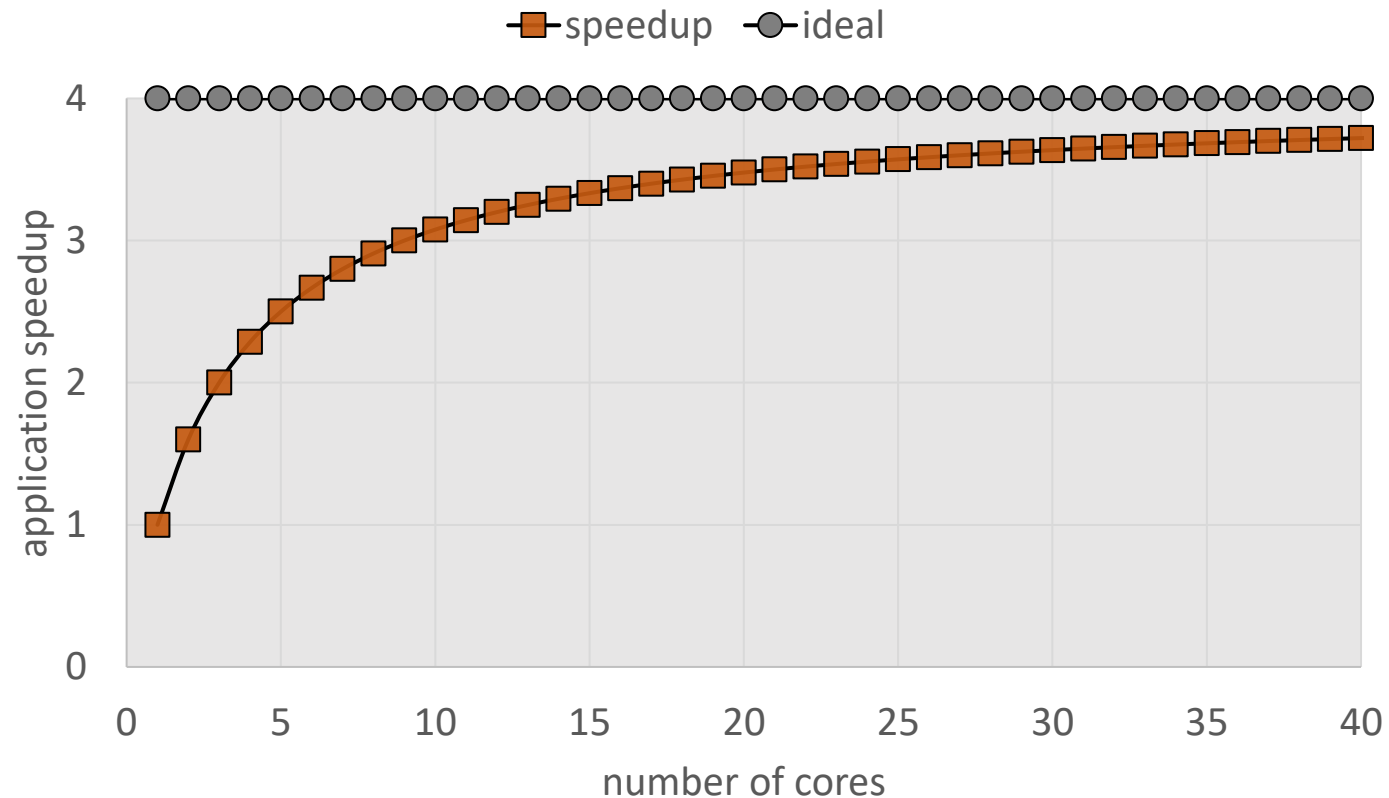
Given an optimization X that accelerates by a factor of S_X and can be applied to $\%_X$ of the application work:

$$\text{Application speedup, } X = AS_X = \frac{1}{(1 - \%_X) + \frac{\%_X}{S_X}}$$

Amdahl's Law – 75% Parallel

$\%_p = 75\%$ of application work

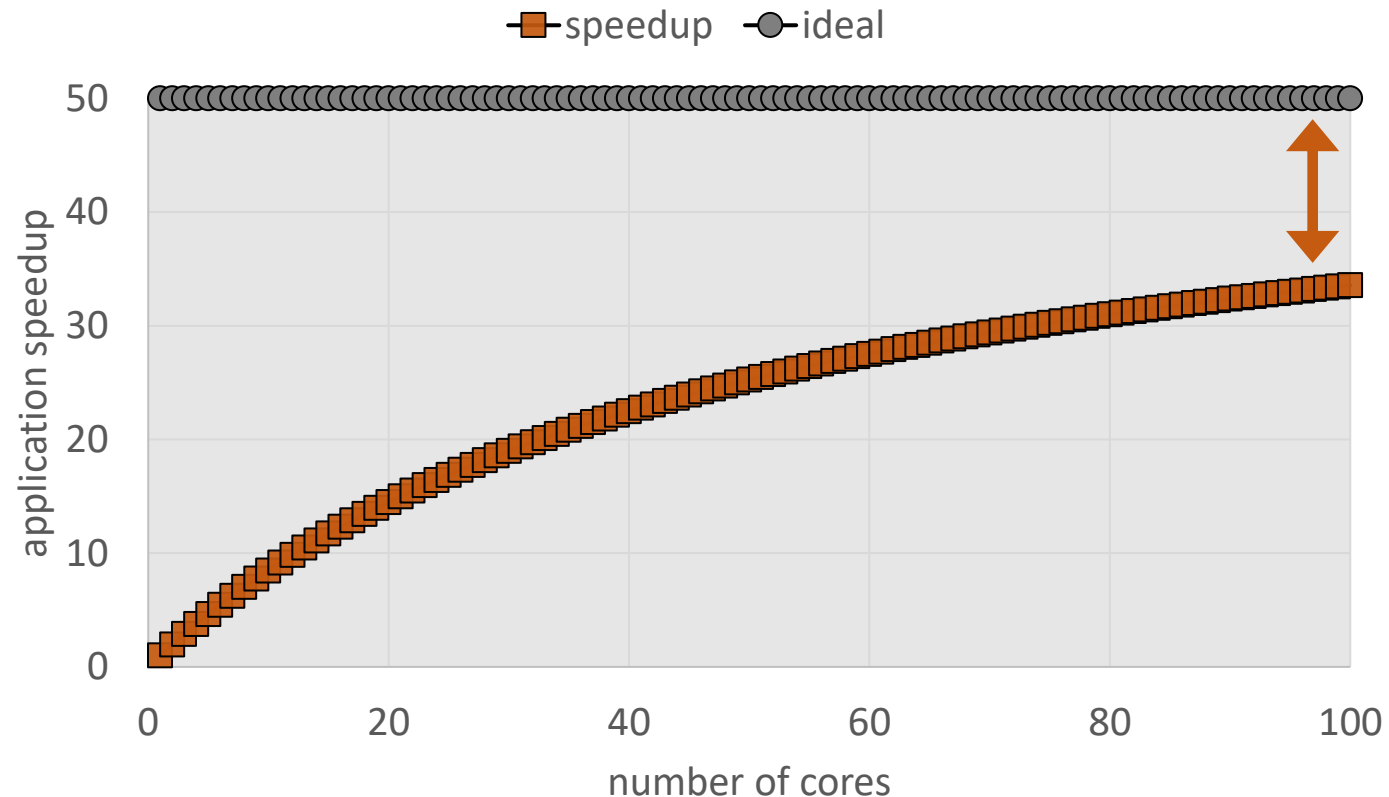
$S_p = P$ processor cores



Amdahl's Law – 98% Parallel

$\%_p = 98\%$ of application work

$S_p = P$ processor cores



Even small percentage of sequential work can greatly limit speedup

Flynn's Taxonomy of Parallel Computers

Michael J. Flynn, *Very High-Speed Computing Systems*, Proc. IEEE, 1966.

SISD (Single Instruction – Single Data)

- One instruction operating on one data location

SIMD (Single Instruction – Multiple Data)

- One instruction operating on multiple data locations

MISD (Multiple Instruction – Single Data)

- Multiple instructions operating on one data location

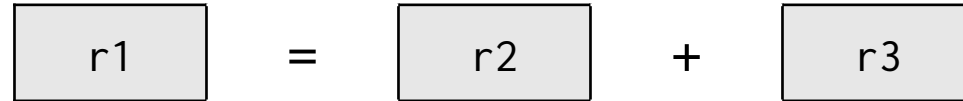
MIMD (Multiple Instruction – Multiple Data)

- Multiple instructions operating on multiple data locations

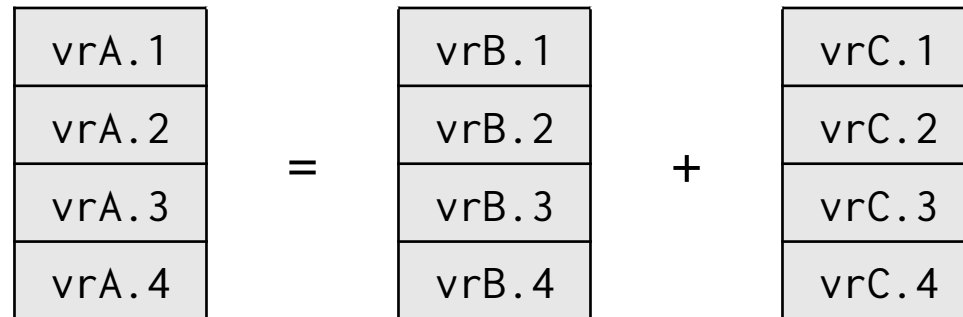
SIMD (Single Instruction – Multiple Data)

Vector (Superword) Instructions:

conventional add r1, r2, r3



vector add vrA, vrB, vrC



Vector (Superword) Instructions

Vector registers are typically composed of multiple smaller registers.

r0	r1	r2	r3	vrA
r4	r5	r6	r7	
r8	r9	r10	r11	vrB
r12	r13	r14	r15	
r16	r17	r18	r19	vrC

e.g., Intel 128-bit Streaming SIMD Extensions (SSE):

- 70 different instructions/operations
- 8 128-bit registers that can be accessed as:
 - Two 64-bit integers or double-precision floating-point
 - Four 32-bit integers or single-precision floating-point
 - Eight 16-bit short integers
 - Sixteen 8-bit chars

Vector (Superword) Instructions

Requirements:

- New instructions in ISA
- More ports in register file
- Register packing/unpacking/shuffling:

$$r5 = r6 + r7$$

$$r3 = r19 + r18$$

$$r8 = r1 + r14$$

$$r9 = r2 + r15$$

r0	r1	r2	r3	vrA
r4	r5	r6	r7	
r8	r9	r10	r11	vrB
r12	r13	r14	r15	
r16	r17	r18	r19	vrC



Vector (Superword) Instructions

Compiler support:

1. Original:

```
a  = ... + d[6*i];  
b  = ... + d[6*i + 4];  
c  = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

Vector (Superword) Instructions

Compiler support:

1. Original:

```
a  = ... + d[6*i];  
b  = ... + d[6*i + 4];  
c  = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

2. Statement Grouping:

```
(  
  a  = ... + d[6*i];  
  b  = ... + d[6*i + 4];  
  c  = ... + d[6*i + 3];  
  ... = ... + a;  
  ... = b / ...;  
  ... = a / ...;  
  ... = c / ...;  
)
```

Vector (Superword) Instructions

Compiler support:

1. Original:

```
a  = ... + d[6*i];  
b  = ... + d[6*i + 4];  
c  = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

2. Statement Grouping:

```
a = ... + d[6*i];  
b = ... + d[6*i + 4];  
c = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

3. Statement Scheduling:

```
a  = ... + d[6*i];  
b  = ... + d[6*i + 4];  
c  = ... + d[6*i + 3];  
... = a / ...;  
... = b / ...;  
... = c / ...;  
... = ... + a;
```

Vector (Superword) Instructions

Compiler support:

1. Original:

```
a = ... + d[6*i];  
b = ... + d[6*i + 4];  
c = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

2. Statement Grouping:

```
a = ... + d[6*i];  
b = ... + d[6*i + 4];  
c = ... + d[6*i + 3];  
... = ... + a;  
... = b / ...;  
... = a / ...;  
... = c / ...;
```

3. Statement Scheduling:

```
a = ... + d[6*i];  
b = ... + d[6*i + 4];  
c = ... + d[6*i + 3];  
... = a / ...;  
... = b / ...;  
... = c / ...;  
... = ... + a;
```

4. Data Layout Optimization:

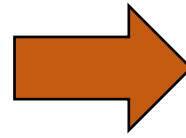
```
a = ... + e[3*i];  
b = ... + e[3*i + 1];  
c = ... + e[3*i + 2];  
... = a / ...;  
... = b / ...;  
... = c / ...;  
... = ... + a;
```



Vector (Superword) Instructions

Loop unrolling:

```
for (i = 0; i < n; i += 1) {  
    a[i] = b[i] + c[i];  
}
```



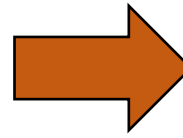
```
for (i = 0; i < n; i += 4) {  
    {  
        a[i]    = b[i]    + c[i];  
        a[i+1]  = b[i+1]  + c[i+1];  
        a[i+2]  = b[i+2]  + c[i+2];  
        a[i+3]  = b[i+3]  + c[i+3];  
    }  
}
```



Vector (Superword) Instructions

Loop unrolling:

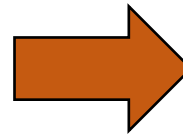
```
for (i = 0; i < n; i += 1) {  
    a[i] = b[i] + c[i];  
}
```



```
for (i = 0; i < n; i += 4) {  
    {  
        a[i]    = b[i]    + c[i];  
        a[i+1]  = b[i+1]  + c[i+1];  
        a[i+2]  = b[i+2]  + c[i+2];  
        a[i+3]  = b[i+3]  + c[i+3];  
    }  
}
```

Loop fusion:

```
for (i = 0; i < n; i += 1)  
    a[i].x = b[i].x + c[i].x;  
...  
for (i = 0; i < n; i += 1)  
    a[i].y = b[i].y + c[i].y;  
...  
for (i = 0; i < n; i += 1)  
    a[i].z = b[i].z + c[i].z;
```

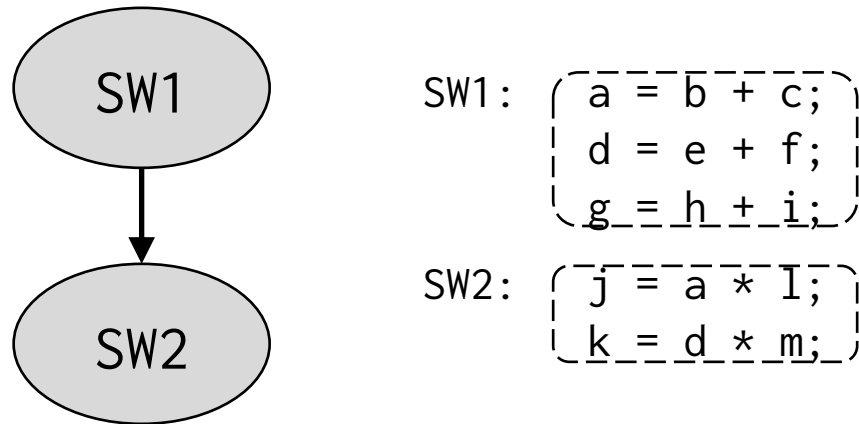


```
for (i = 0; i < n; i += 1) {  
    {  
        a[i].x = b[i].x + c[i].x;  
        a[i].y = b[i].y + c[i].y;  
        a[i].z = b[i].z + c[i].z;  
    }  
}
```



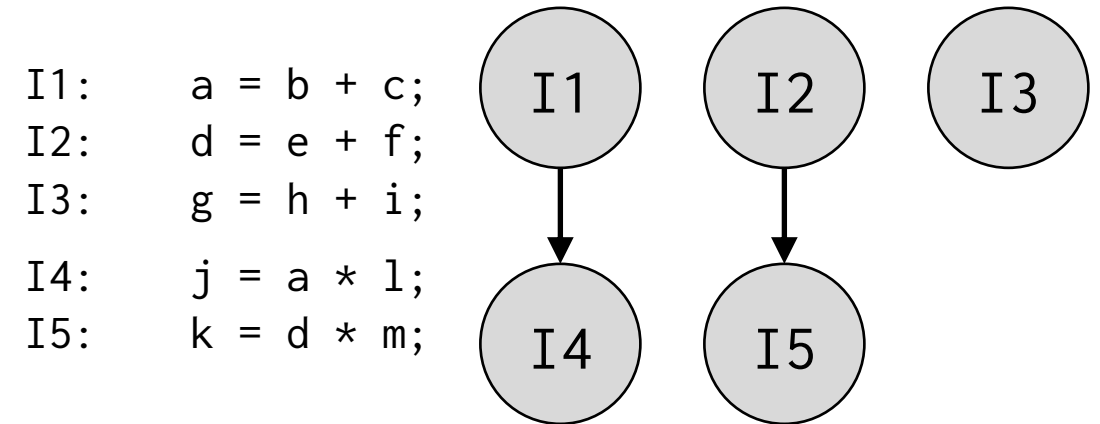
Vector (Superword) Instructions

superword-level parallelism



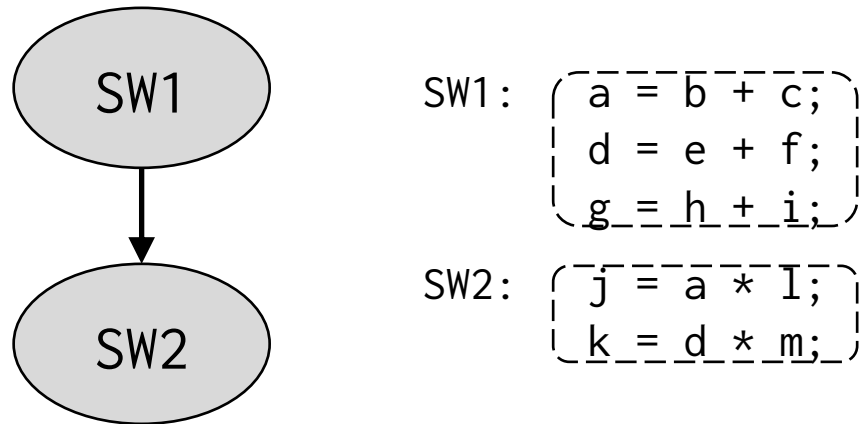
VS

instruction-level parallelism



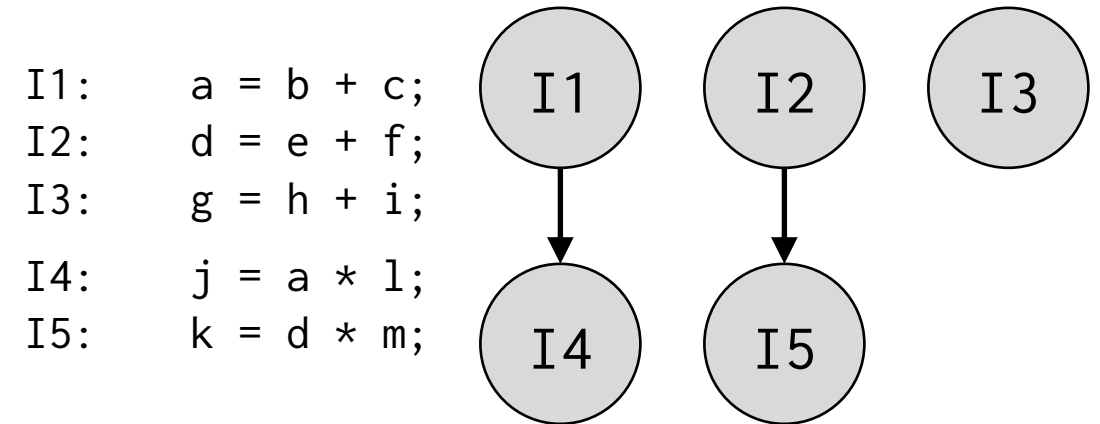
Vector (Superword) Instructions

superword-level parallelism



VS

instruction-level parallelism

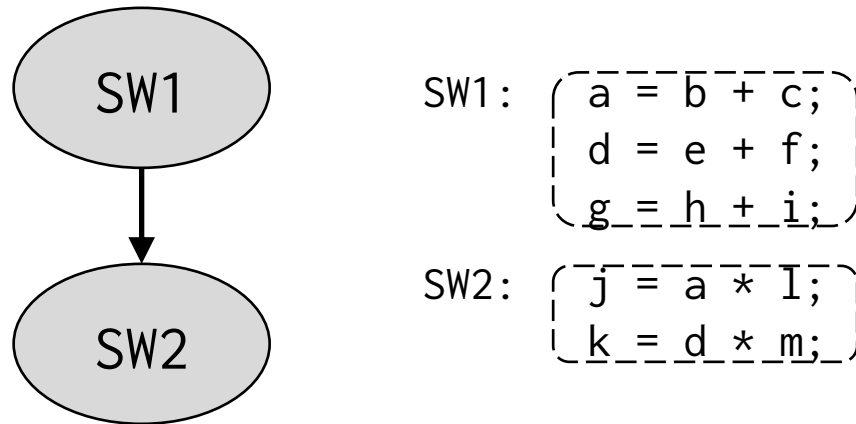


- Needs register packing/unpacking
- Less flexible (e.g., same operations)

- More dynamic instructions issued
- Superscalar cost for same performance

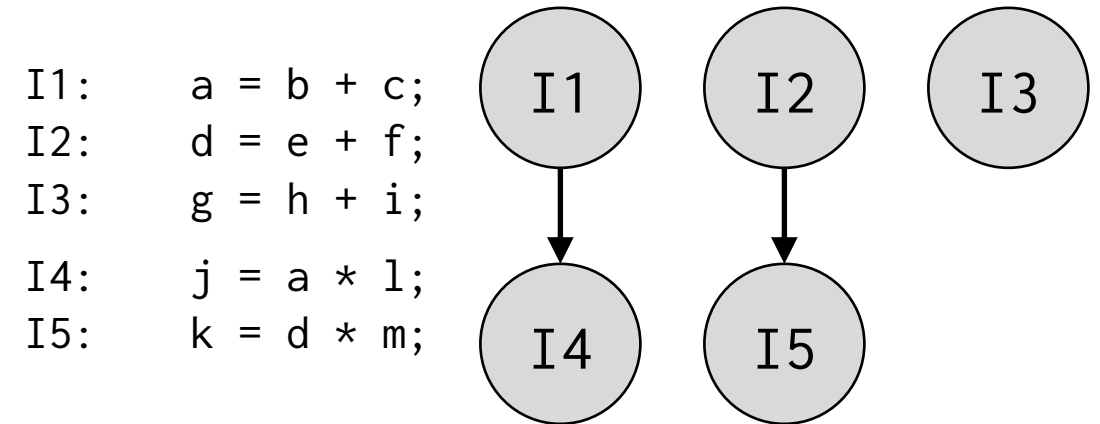
Vector (Superword) Instructions

superword-level parallelism



VS

instruction-level parallelism



An analogy:

- Needs to be a matching (or similar) item to be discounted
- BOGO less flexible (infrequent, same item), but membership has high cost.

Flynn's Taxonomy of Parallel Computers

Michael J. Flynn, *Very High-Speed Computing Systems*, Proc. IEEE, 1966.

SISD (Single Instruction – Single Data)

- One instruction operating on one data location

SIMD (Single Instruction – Multiple Data)

- One instruction operating on multiple data locations

MISD (Multiple Instruction – Single Data)

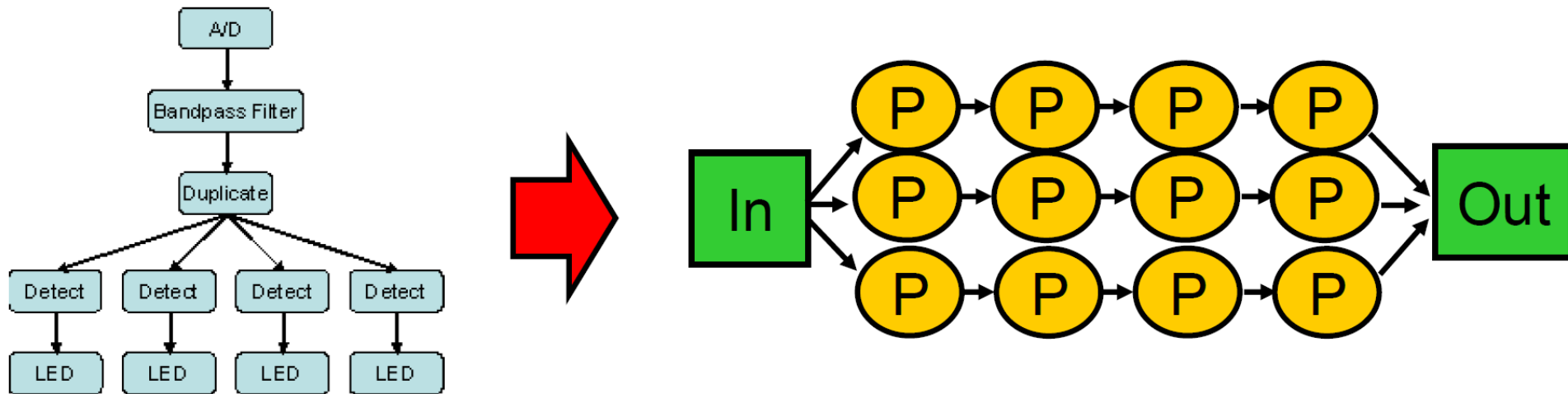
- Multiple instructions operating on one data location

MIMD (Multiple Instruction – Multiple Data)

- Multiple instructions operating on multiple data locations

MISD (Multiple Instruction – Single Data)

Streaming Architectures:



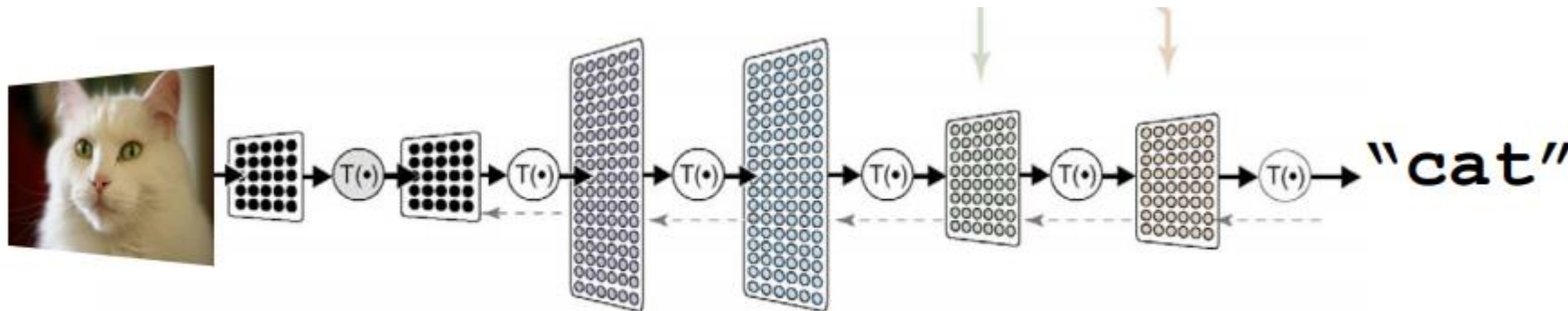
e.g., Streaming big data analytics:

- IBM stream computing
- Apache Spark streaming

Systolic Array

Data from memory is streamed into a pipelined array of processing nodes, where each node processes its operands and *pulses* (hence, systolic) them down the pipeline.

- Proposed in the 1980s (H. T. Kung, CMU WARP project)
- Used commonly in machine learning hardware accelerators today (e.g., Google's TPU)

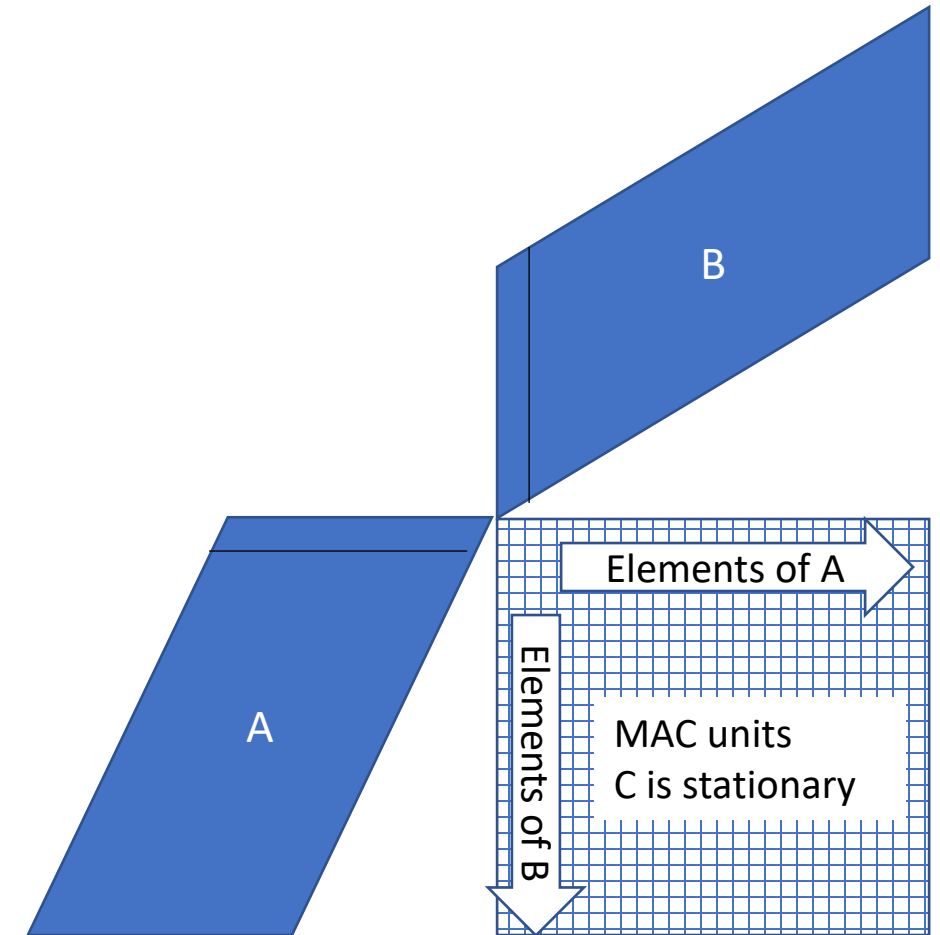
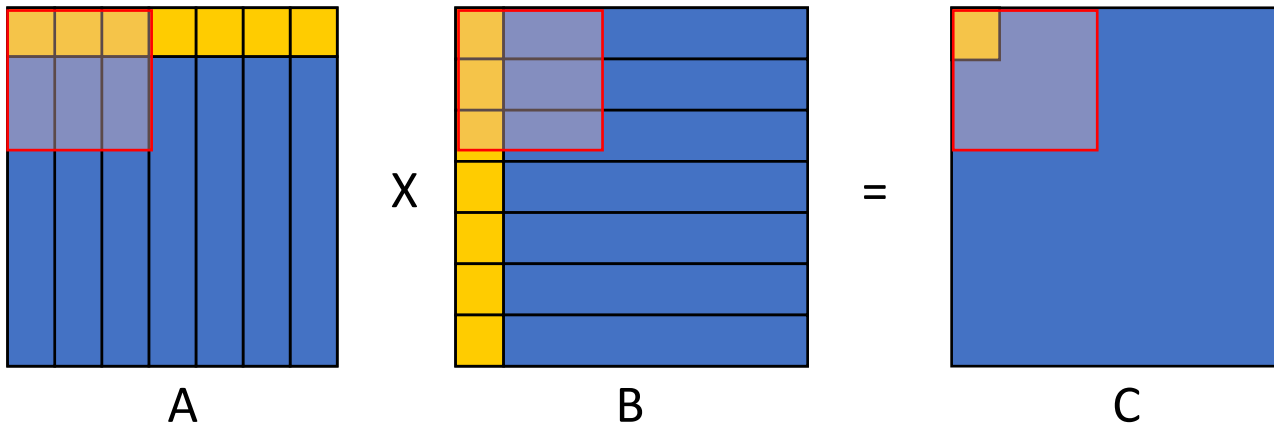


Slide from "Large-Scale Deep Learning with TensorFlow for Building Intelligent Systems," by Jeff Dean, ACM Webinar, 7/7/16

Systolic Array

Systolic arrays for (tilted) matrix multiplication:

- Multiply-accumulate (MAC) units organized as a 2D array
- Input matrices A and B enter diagonally and accumulate each entry of C at each MAC unit register
- Will revisit later in course



MIMD (Multiple Instruction – Multiple Data)

Multiple processors that can operate independently yet can share resources and communicate with each other

Multiple processors in a core:

- Connected via microarchitecture
- Simultaneous multithreading (SMT), hyperthreading

Multiple processors in a chip:

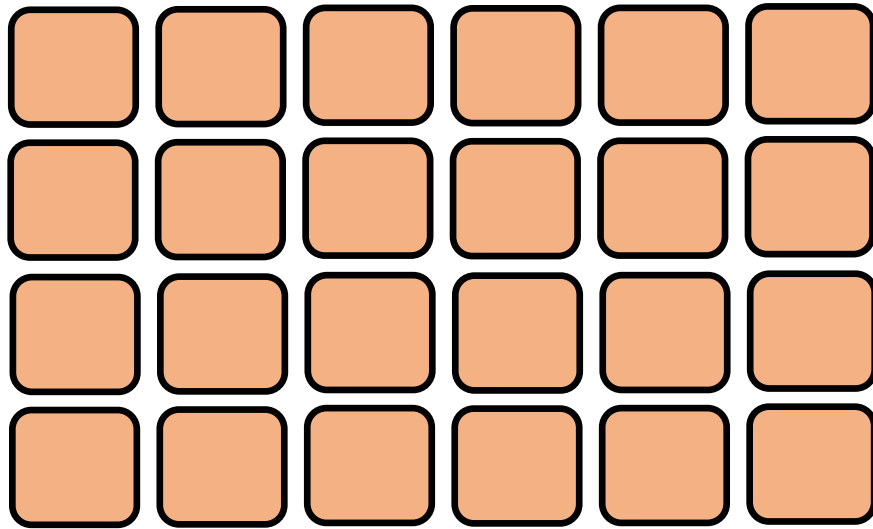
- Connected via on-chip interconnect/memory hierarchy
- Chip multiprocessors (CMPs), shared-memory multiprocessors

Multiple processors in a cluster:

- Connected via IO/internetworking
- Datacenters, warehouse-scale computers

Multithreaded and Multiprogrammed

multithreaded workload

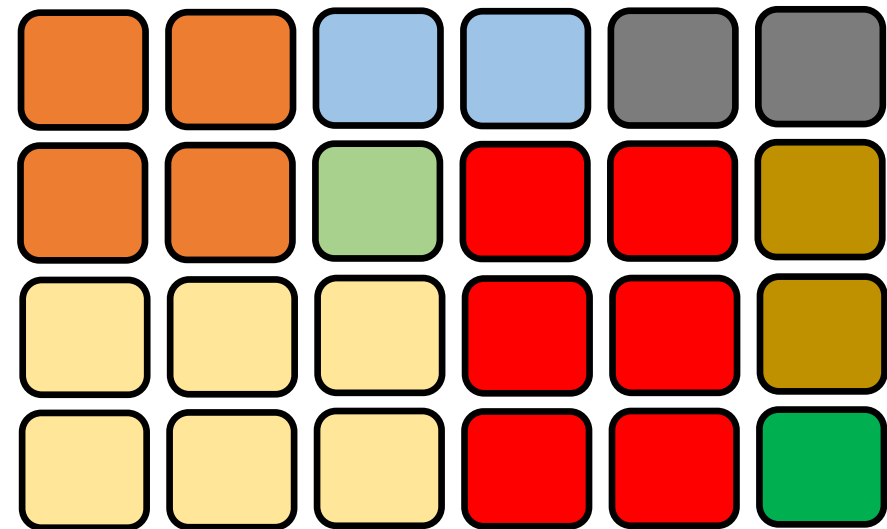


threads of same program

Challenges:

- Communication locality
- Thread synchronization
- Load imbalance

multiprogrammed workload

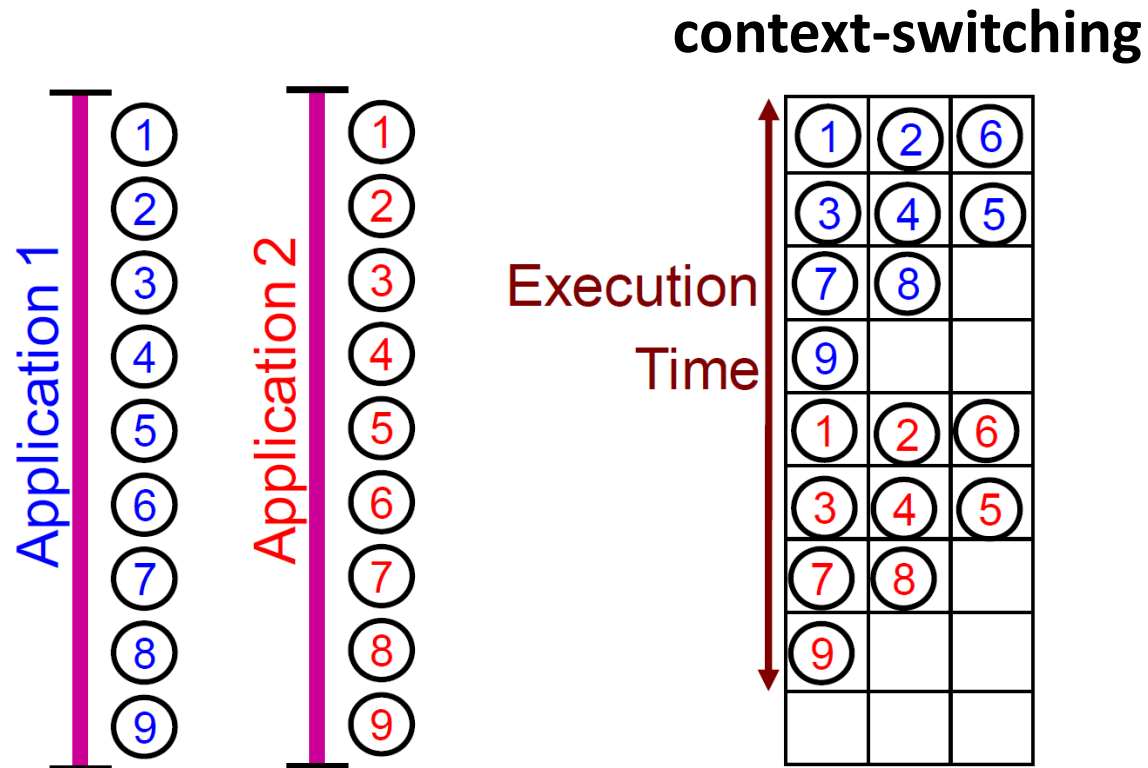


threads of different programs

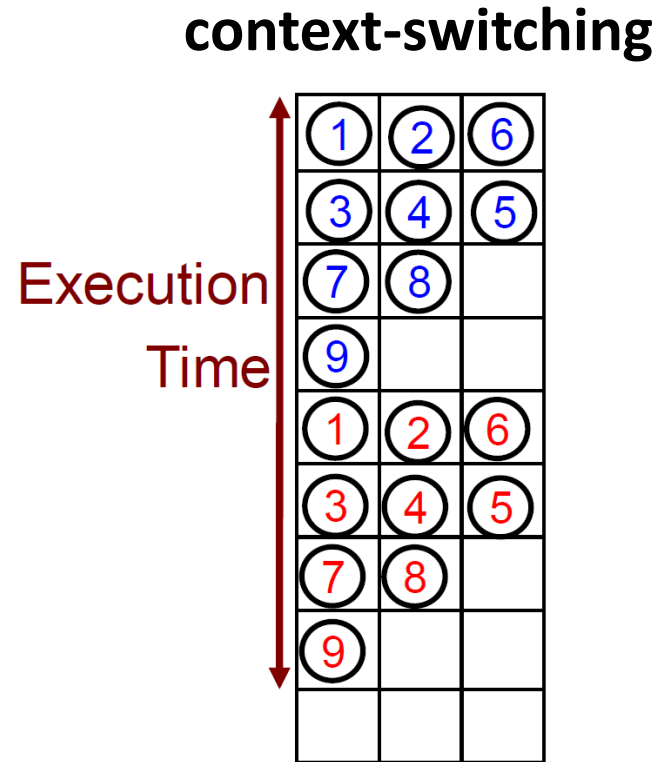
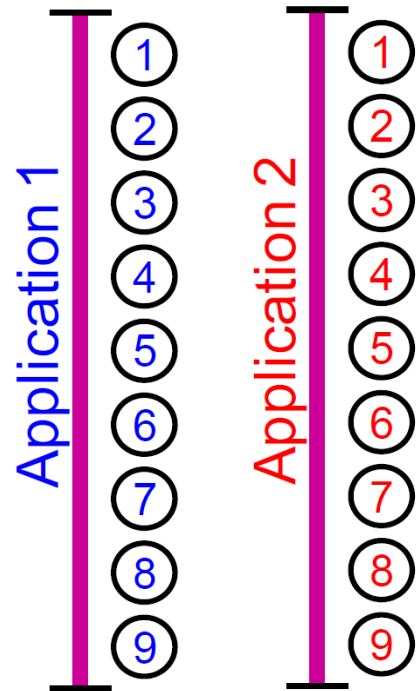
Challenges:

- Resource allocation/arbitration
- Scheduling and fairness

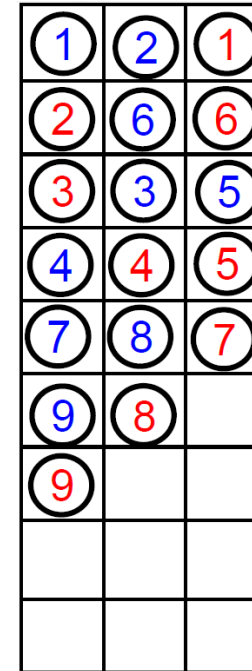
Multithreading



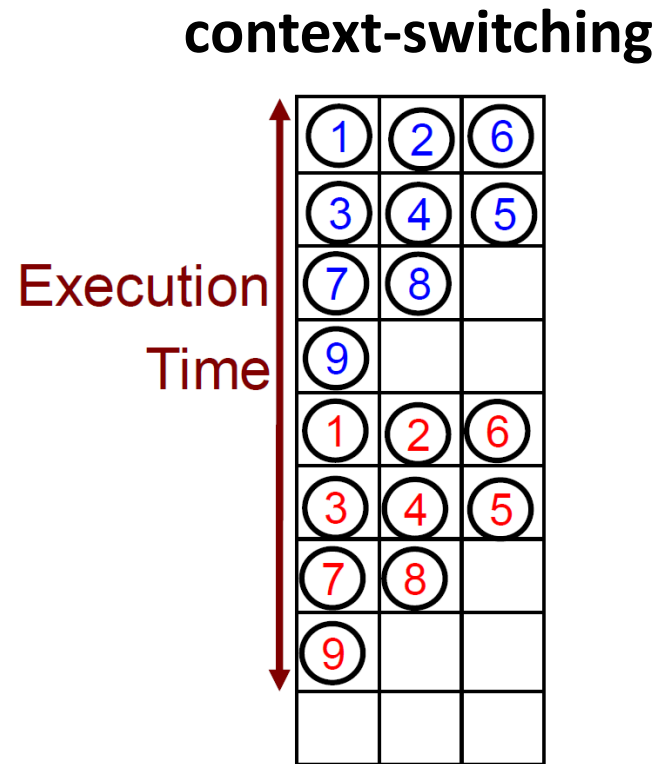
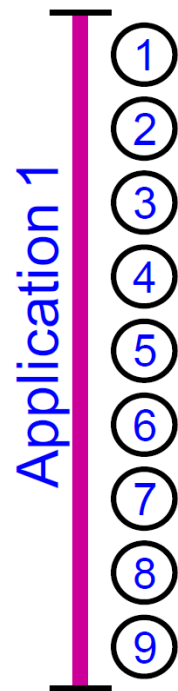
Multithreading



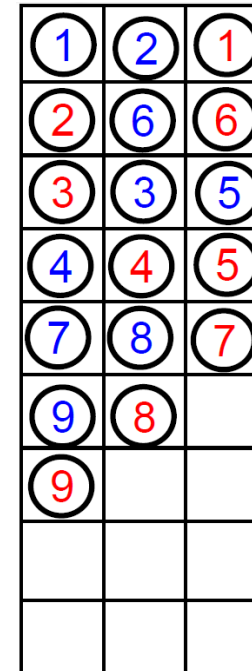
simultaneous
multithreading



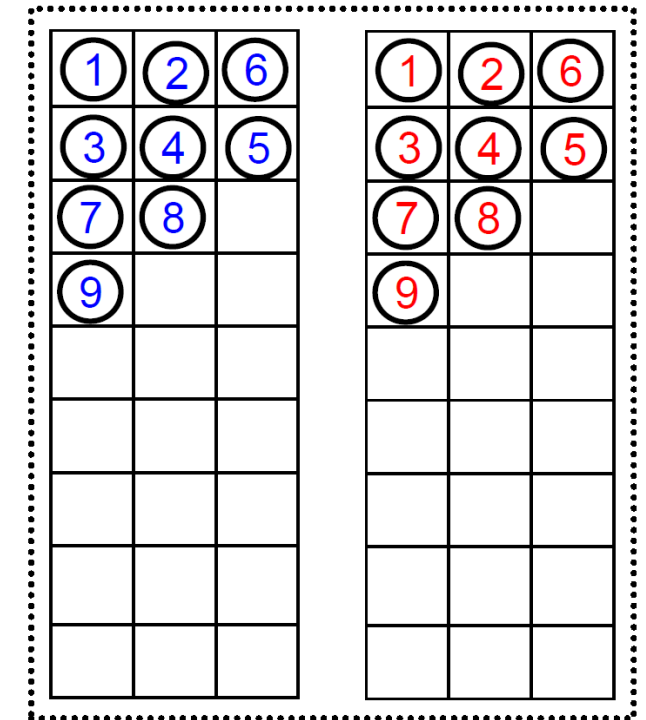
Multithreading



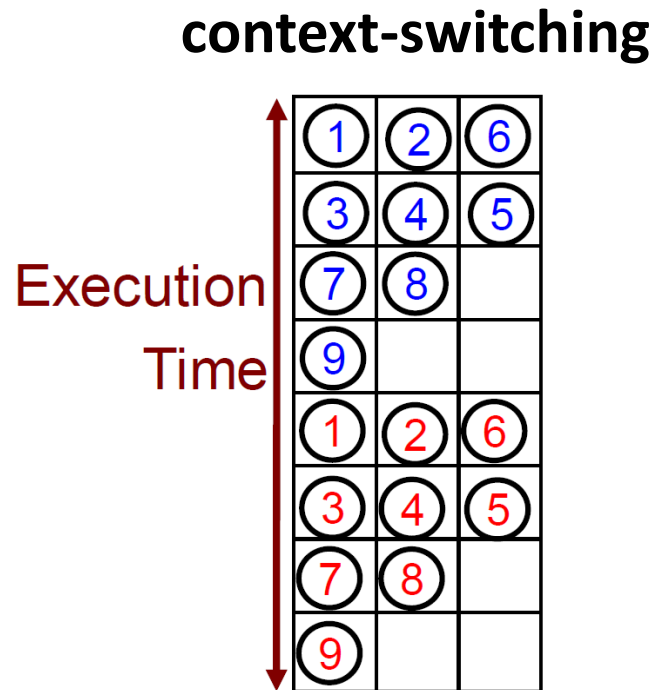
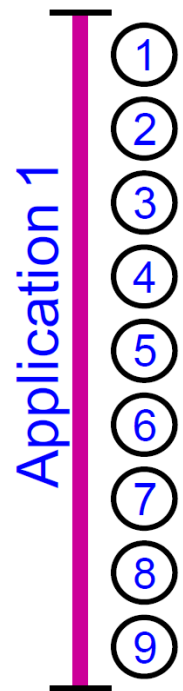
simultaneous
multithreading



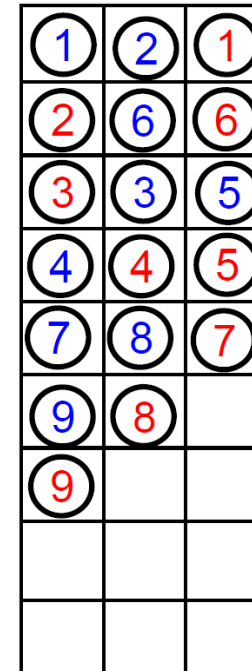
chip multiprocessor



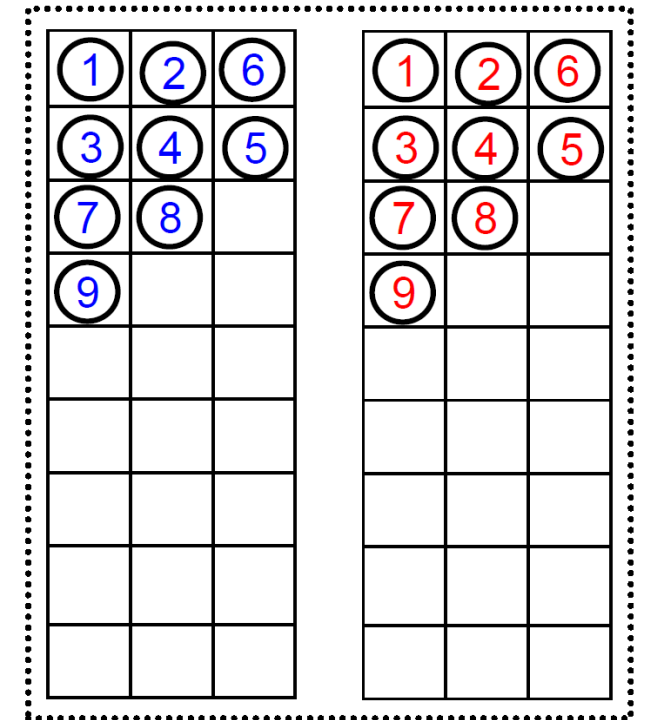
Multithreading



simultaneous multithreading



chip multiprocessor



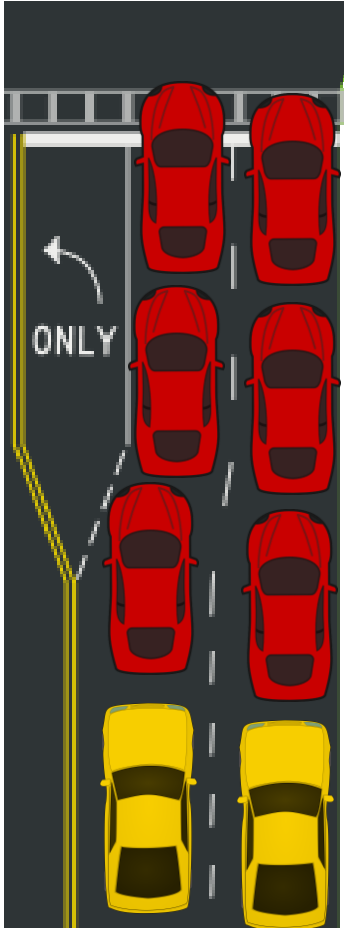
parallelism/throughput

area/complexity



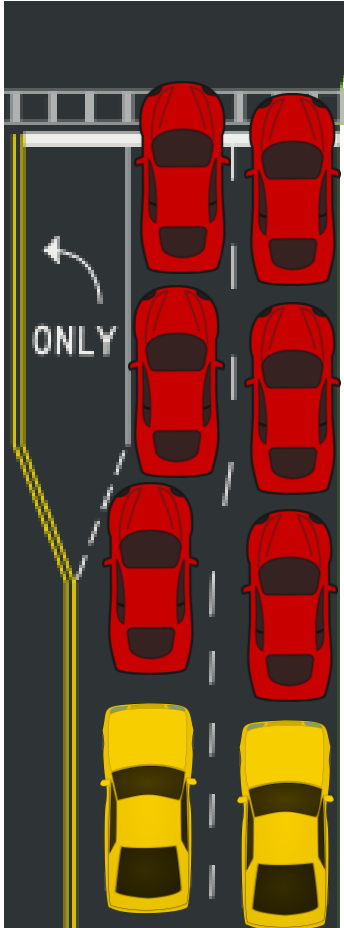
Multithreading – An Analogy

convoying

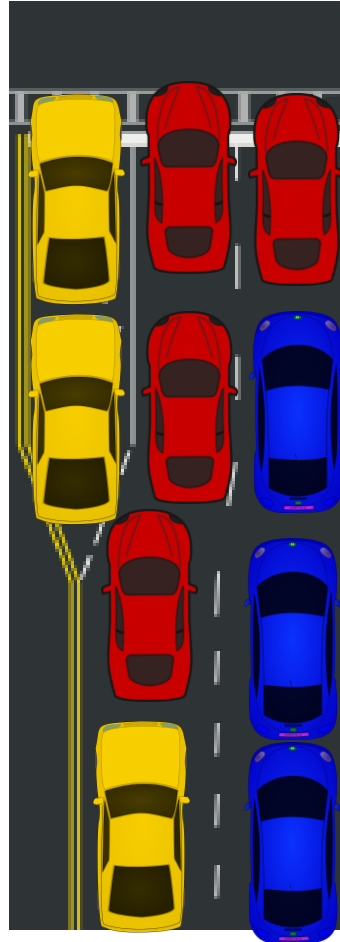


Multithreading – An Analogy

convoying

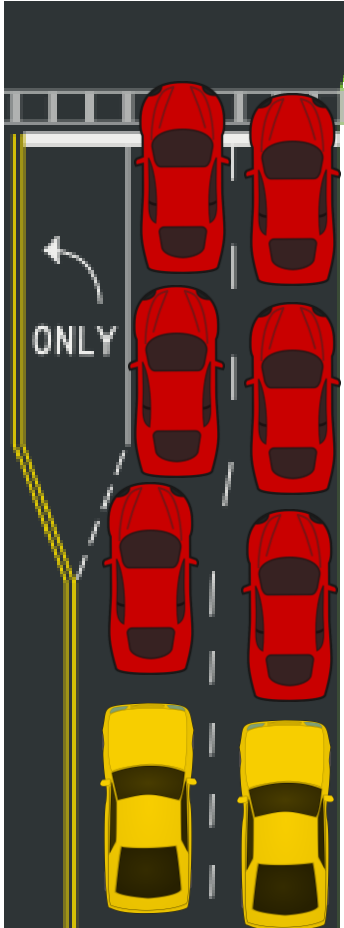


self-navigating

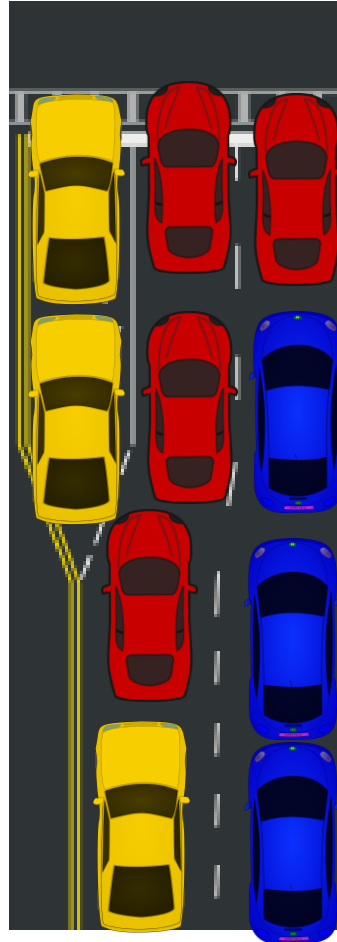


Multithreading – An Analogy

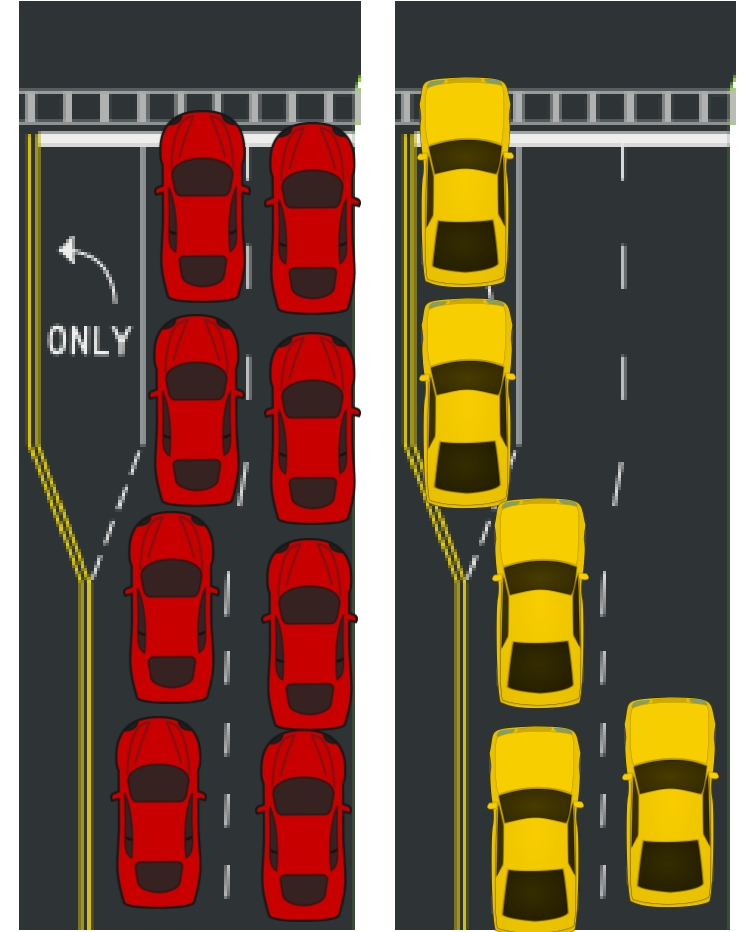
convoying



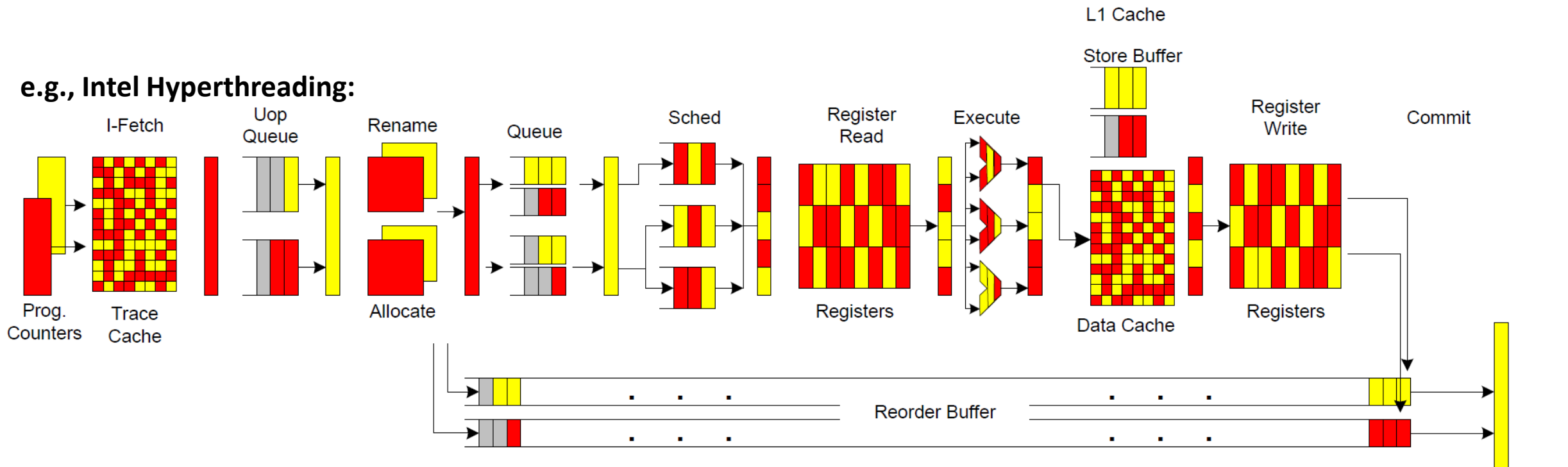
self-navigating



separate routes



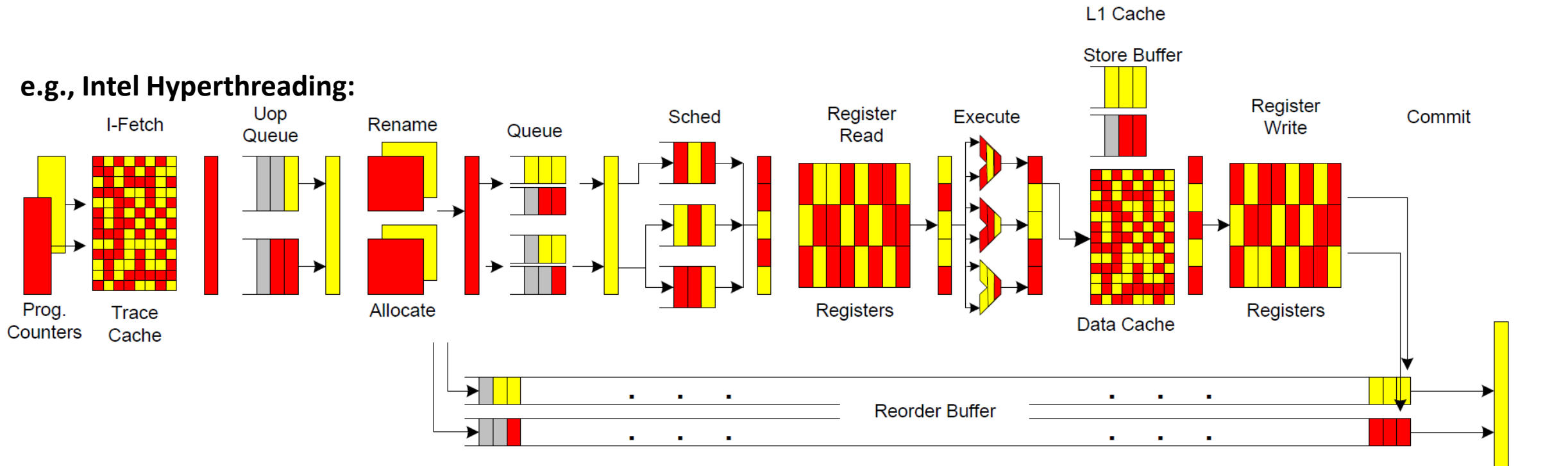
Simultaneous Multithreading



Simultaneous Multithreading

Design decisions:

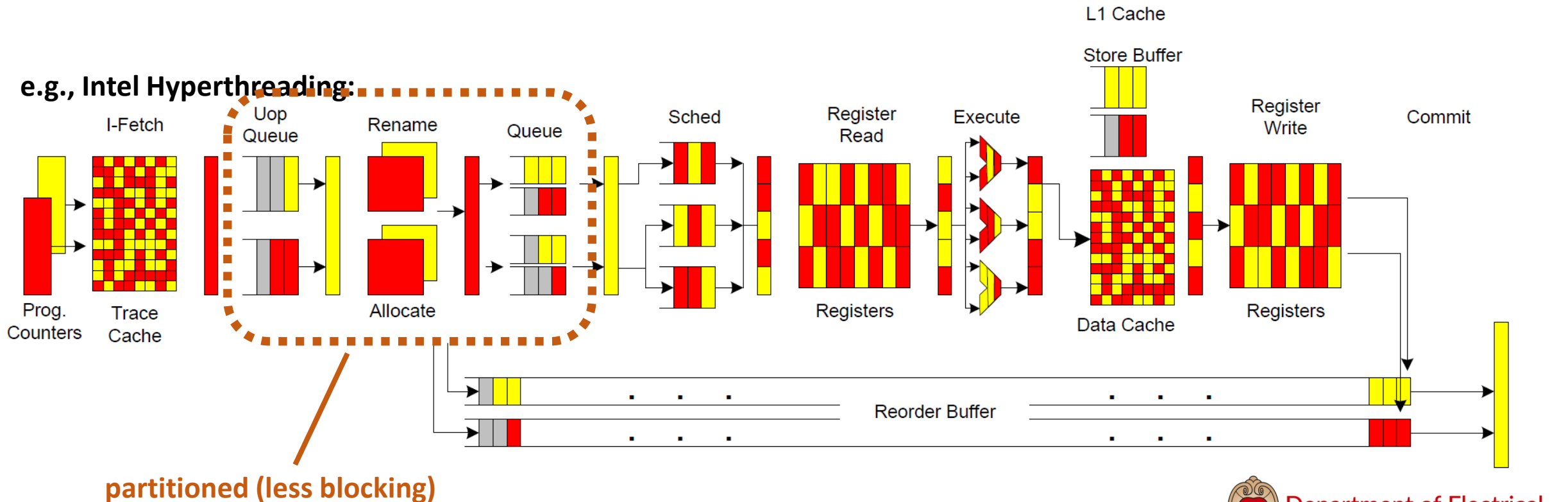
- Partitioned or pooled pipeline units?



Simultaneous Multithreading

Design decisions:

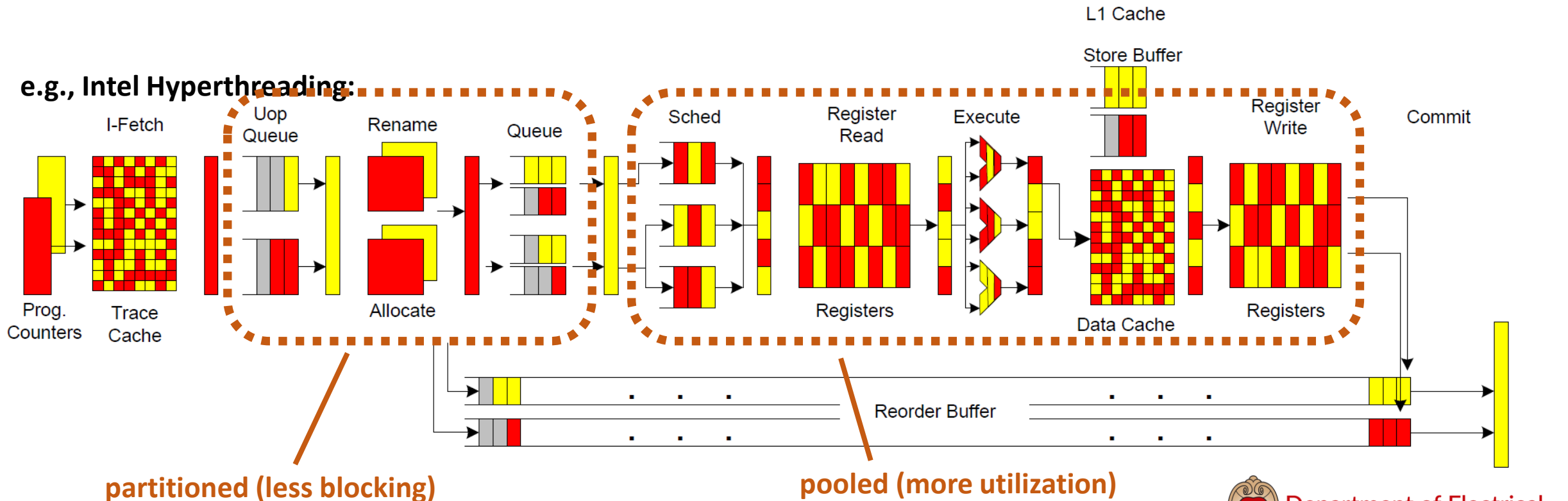
- Partitioned or pooled pipeline units?



Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?

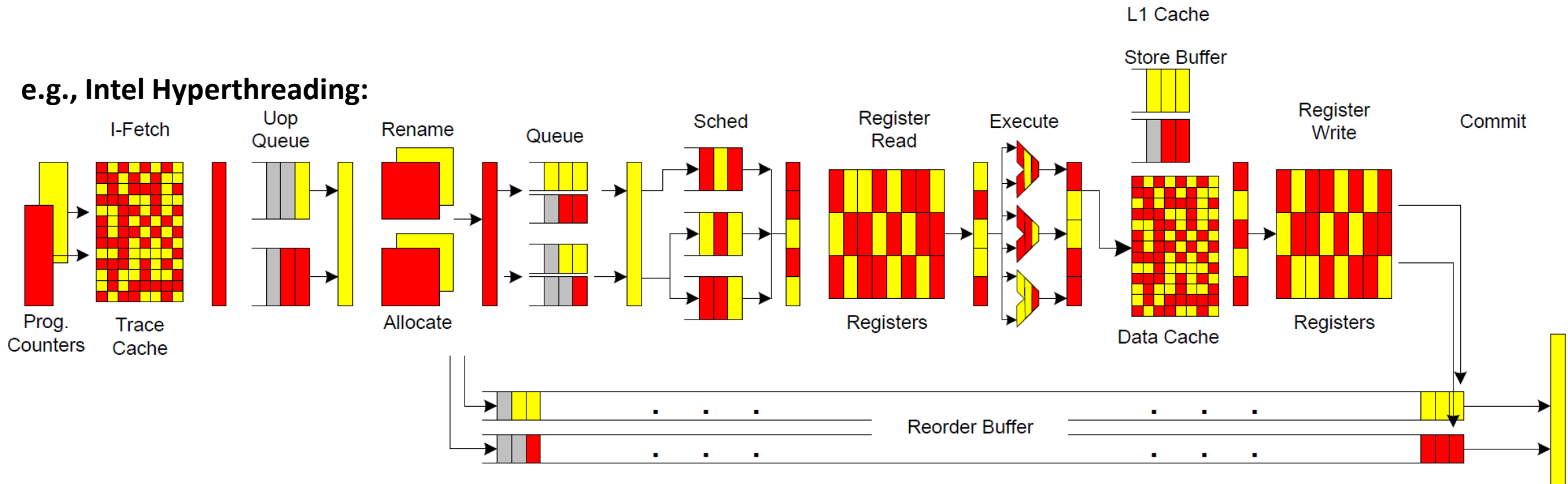


Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?

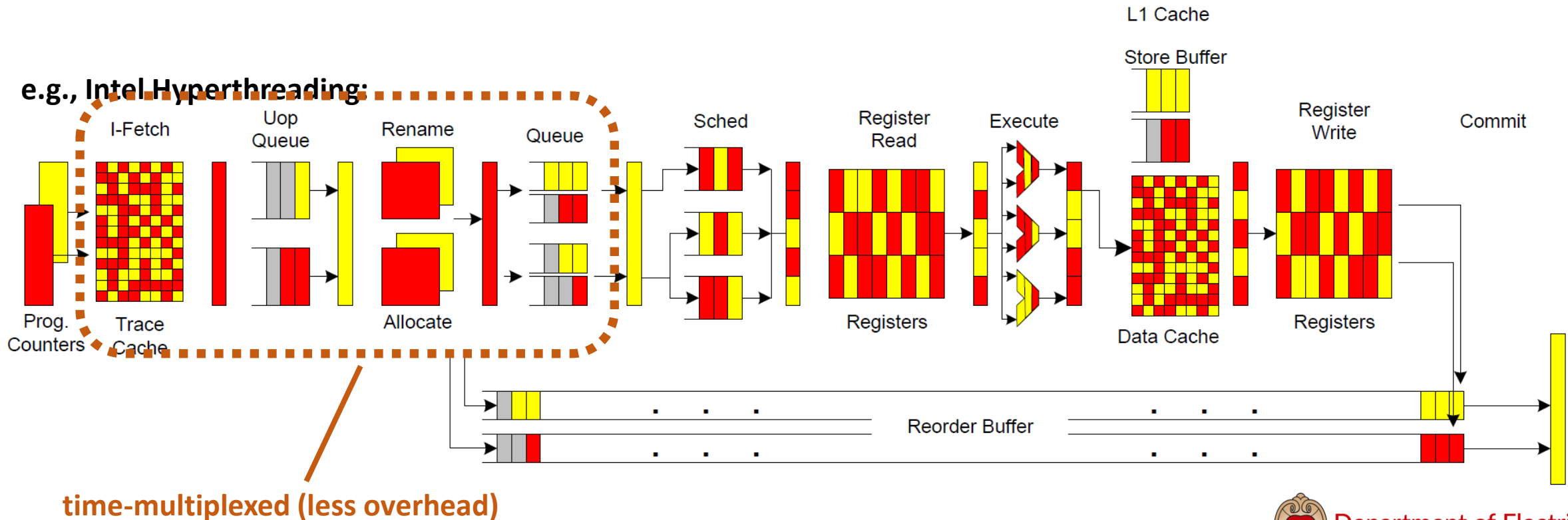
e.g., Intel Hyperthreading:



Simultaneous Multithreading

Design decisions:

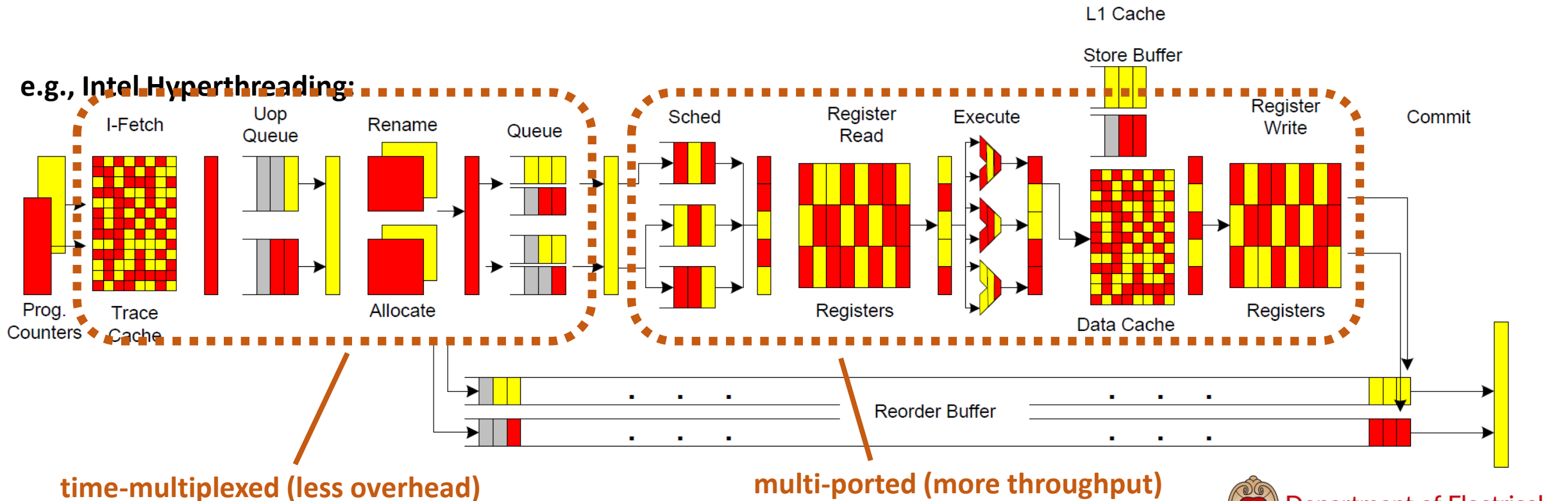
- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?



Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?

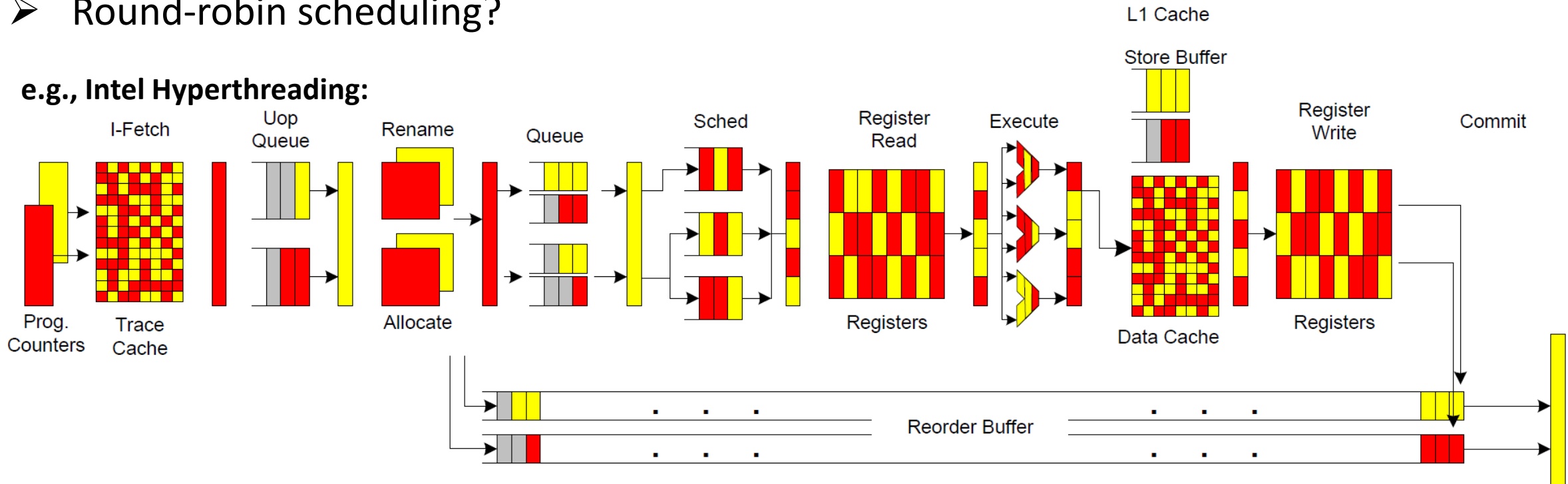


Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?
- Round-robin scheduling?

e.g., Intel Hyperthreading:

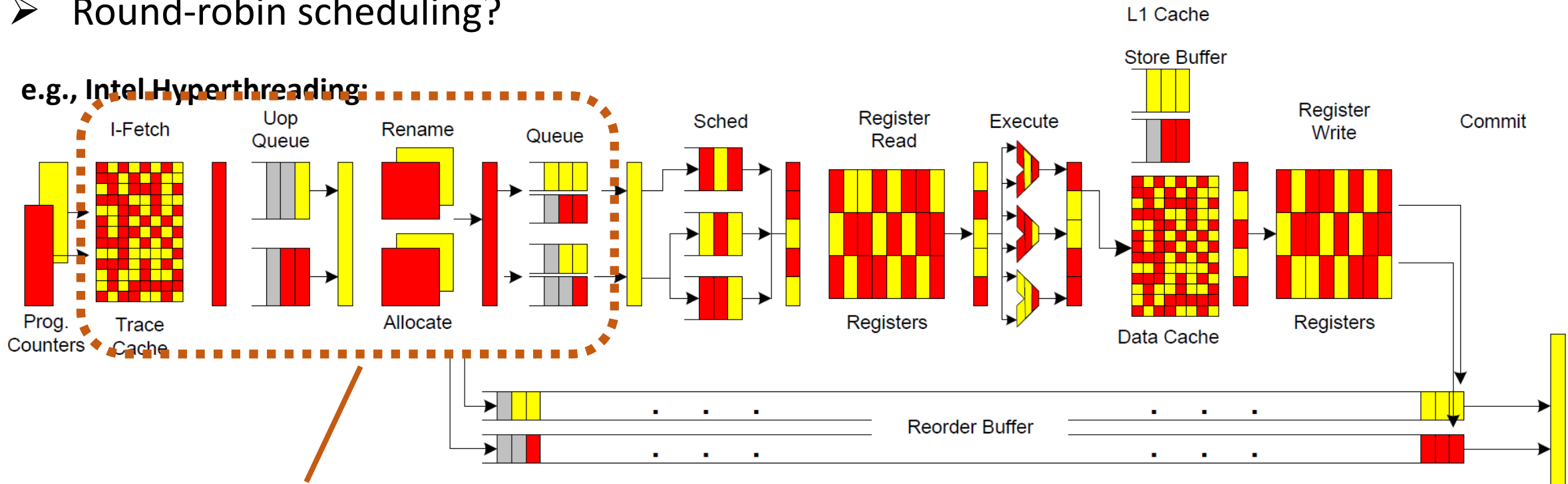


Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?
- Round-robin scheduling?

e.g., Intel Hyperthreading:



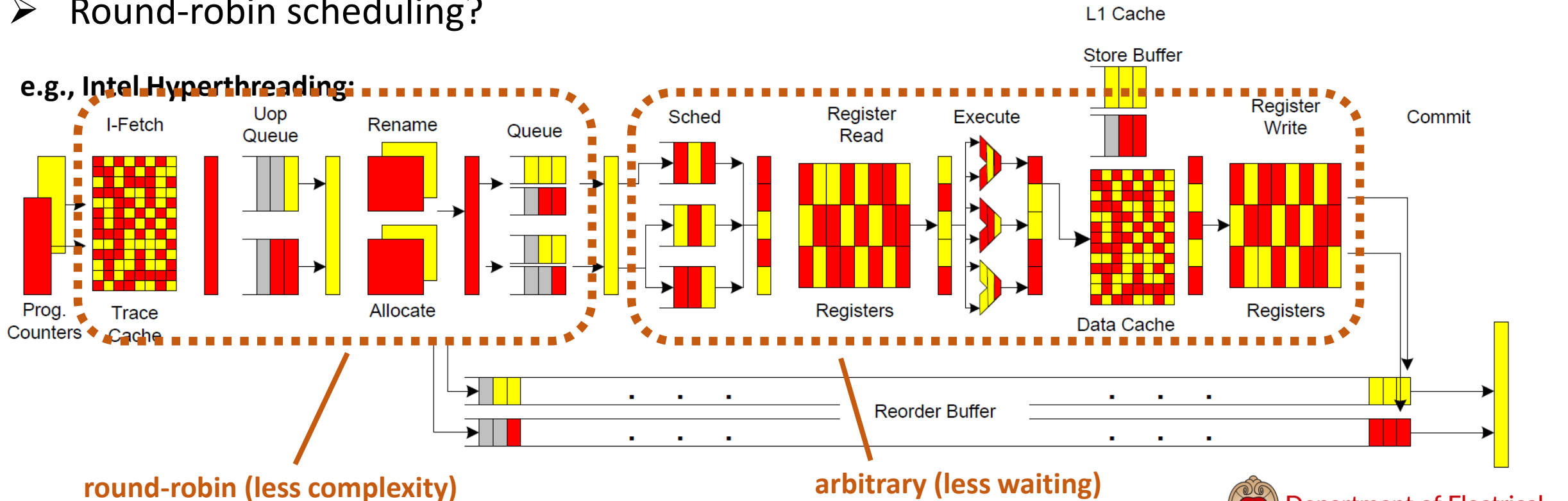
round-robin (less complexity)

Simultaneous Multithreading

Design decisions:

- Partitioned or pooled pipeline units?
- Time-multiplexed or multi-ported/replicated?
- Round-robin scheduling?

e.g., Intel Hyperthreading:



Parallel Programming

Key design decisions:

Extracting parallelism/concurrency

- Pthreads, OpenMP

Communicating data

- Shared memory, message passing

Synchronizing computations

- Locks, barriers

POSIX Threads (pthreads)

Popular library for explicitly creating and managing threads in a process.

```
void *thread(void *vargp) {  
    printf("Hello, world!\n");  
    return NULL;  
}  
  
int main() {  
    pthread_t tid;  
    pthread_create(&tid, NULL, thread, NULL);  
    pthread_join(tid, NULL);  
    return 0;  
}
```

POSIX Threads (pthreads)

Popular library for explicitly creating and managing threads in a process.

```
void *thread(void *vargp) {  
    printf("Hello, world!\n");  
    return NULL;  
}  
  
int main() {  
    pthread_t tid;  
    pthread_create(&tid, NULL, thread, NULL);  
    pthread_join(tid, NULL);  
    return 0;  
}
```

start executing thread

wait for thread to finish

POSIX Threads (pthreads)

Popular library for explicitly creating and managing threads in a process.

```
void *thread(void *vargp) {  
    printf("Hello, world!\n");  
    return NULL;  
}
```

```
int main() {  
    pthread_t tid;  
    pthread_create(&tid, NULL, thread, NULL);  
    pthread_join(tid, NULL);  
    return 0;  
}
```

Annotations for `pthread_create` and `pthread_join`:

- `tid`: get new thread's ID
- `NULL` (first): set thread attributes (NULL = default)
- `thread`: set thread function to execute
- `NULL` (second): set thread function arguments (void *)
- `NULL` (in `pthread_join`): get thread's return value (void **)

POSIX Threads (pthreads)

Static threads:

One master creates and aggregates threads (e.g., scientific workloads, linear algebra)

Dynamic threads:

Created threads can create other threads (e.g., recursion, graph/tree workloads)

Pool of threads:

Statically-created threads pop from queue of tasks (e.g., queries, low-overhead dynamic)

Pipelined threads:

Statically-created producer-consumer threads (e.g., streaming workloads)

POSIX Threads (pthreads)

Flexible but **error-prone** for novice programmers:

- Multithreading introduces the aspect of “my code waiting for my code”; deadlocks and livelocks are born
- A bug in one small part of the code (e.g., memory leak, I/O dump) can really blow up quickly
- Non-deterministic execution makes it hard to find timing and synchronization errors (i.e., race conditions); reproducing and fixing these bugs is difficult since debuggers can often intrude in the timing (i.e., the observer effect)

OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

```
for (int i = 0; i < N; i++) {  
    A[i] = 0;  
}
```

OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

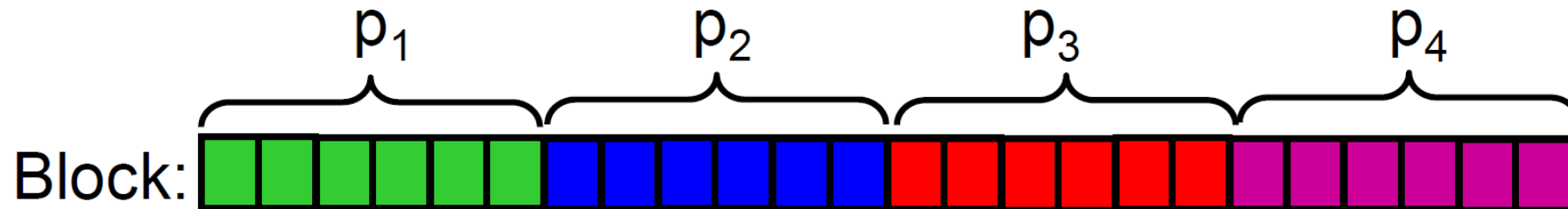
```
#pragma omp parallel for  
for (int i = 0; i < N; i++) {  
    A[i] = 0;  
}
```

OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    A[i] = 0;
}
```

static block scheduling

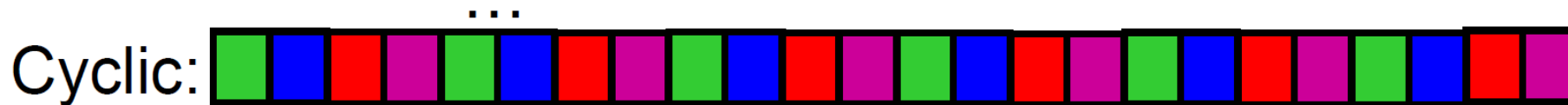


OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

```
#pragma omp parallel for schedule(static,1)
for (int i = 0; i < N; i++) {
    A[i] = 0;
}
```

static cyclic scheduling

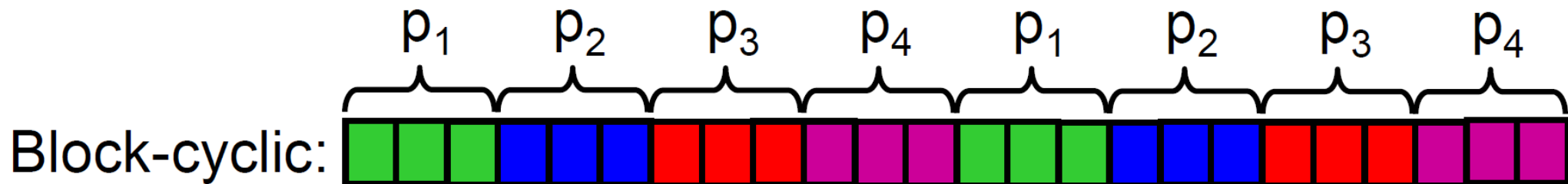


OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

```
#pragma omp parallel for schedule(static,3)
for (int i = 0; i < N; i++) {
    A[i] = 0;
}
```

static block-cyclic scheduling

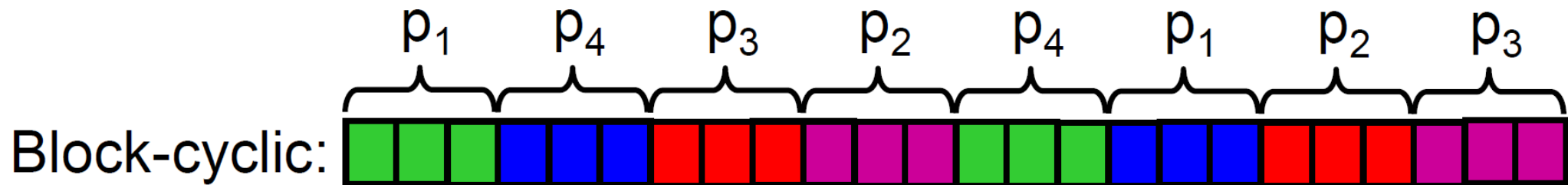


OpenMP

Simple parallelization API for common, regular code structures (e.g., loops).

```
#pragma omp parallel for schedule(dynamic,3)
for (int i = 0; i < N; i++) {
    A[i] = 0;
}
```

dynamic block-cyclic scheduling



Parallel Loops

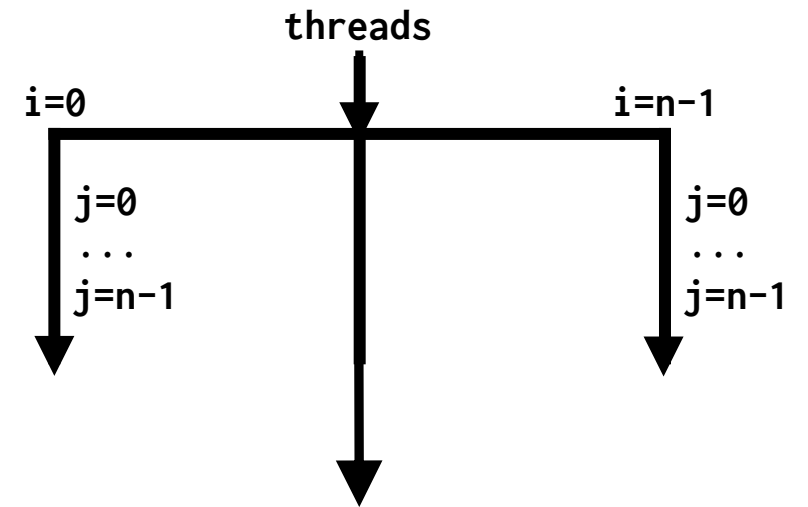
Parallelizing outer vs. inner loops:

```
for(int i = 0; i < n; i++) {  
    for(int j = 0; j < n; j++) {  
        b[i][j] = a[i][j];  
    }  
}
```

Parallel Loops

Parallelizing outer vs. inner loops:

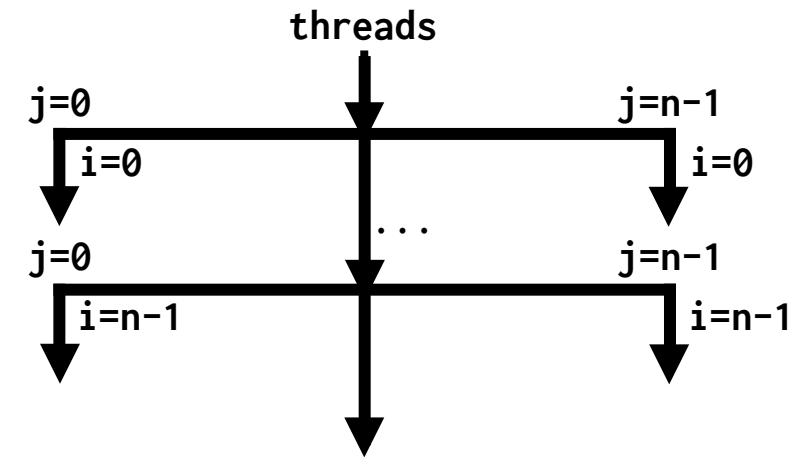
```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        b[i][j] = a[i][j];
    }
}
```



Parallel Loops

Parallelizing outer vs. inner loops:

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for  
    for(int j = 0; j < n; j++) {  
        b[i][j] = a[i][j];  
    }  
}
```



Loop Dependences

Are any of these loops parallelizable?

```
for(int i = 0; i < n; i++) {  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

Are any of these loops parallelizable?

```
#pragma omp parallel for?  
for(int i = 0; i < n; i++) {  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

Are any of these loops parallelizable?

```
for(int i = 0; i < n; i++) {  
    #pragma omp parallel for?  
    for(int j = 2; j < n; j++) {  
        b[i][j] = a[i][j-2];  
        a[i][j] = c[i][j];  
    }  
}
```

Loop Dependences

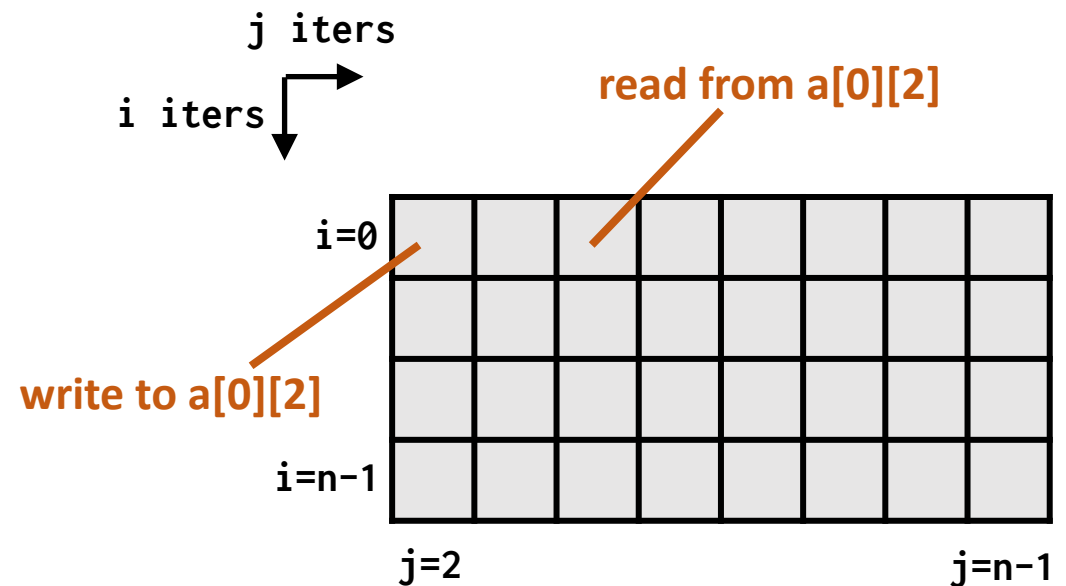
Are any of these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```

Loop Dependences

Are any of these loops parallelizable?

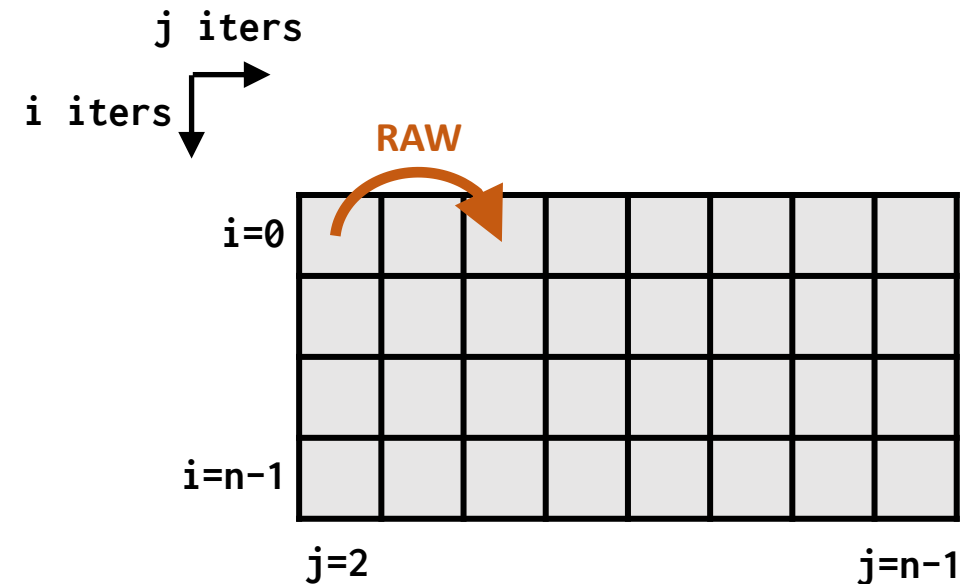
```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```



Loop Dependences

Are any of these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```



Loop Dependences

Are any of these loops parallelizable?

```
#pragma omp parallel for
for(int i = 0; i < n; i++) {
    #pragma omp parallel for
    for(int j = 2; j < n; j++) {
        b[i][j] = a[i][j-2];
        a[i][j] = c[i][j];
    }
}
```

